Lab Assignment 7 — Multiobjective Optimization

37 algorithms, trade-offs and applications

Student	Manavaditya Rajesh
Registration number	241070040 (roll number R = 40)
Deep-dive algorithms (Task C)	Alg. 1 = ((40-1) mod 37)+1 = **3 Benson's Method**; Alg. 2 = ((40+11) mod 37)+1 = **15 NSGA-III**; Alg. 3 = ((40+23) mod 37)+1 = **27 MODE**
Language / tools	Python 3 with NumPy and Matplotlib only for all 37 algorithms and the toolkit; scipy.optimize / linprog only as sub-problem solver of algorithms 1–7 (Section 2A exception); scikit-learn only to load the Wisconsin data (P9); reportlab only to typeset this PDF
Budget / seeds	10,000 function evaluations per run (1,500 for P9 because every evaluation is a 5-fold 5-NN cross-validation); seed 1 for Task A; seeds 1–10 for Tasks C and D; mean ± std reported

Reading guide. Section 1 states the 14 problems; Section 2 the toolkit and its unit tests; Section 3 (Task A) the implementation log and one figure per algorithm; Section 4 (Task B) the master comparison table; Section 5 (Task C) the deep dives; Section 6 (Task D) the head-to-head experiments; Section 7 (Task E) algorithm selection; Section 8 the answers to every 'Investigate' question; Section 9 (Task F) viva preparation; Section 10 reproducibility. Figures are referred to as Fig. A-nn (Task A, algorithm nn), Fig. C-n, Fig. D-n, Fig. E-n. All numbers come from the raw files in the outputs archive.

Honest notes on scope. (i) The 37 algorithms are grouped into 11 Python modules by family (each algorithm is its own class with *initialize / evaluate / iterate / run* and a docstring listing the lecture steps) rather than 37 separate files. (ii) I did not have the lecture slides, so the 'car Pareto-front example' and 'Deb's constrained-dominance table' used in the unit tests are small examples I constructed with the same logic. (iii) Where an algorithm behaves poorly on a problem (VEGA and MODE collapse on ZDT2, NSGA-III is under-converged at 10,000 evaluations on DTLZ2) I report it as measured and explain why, rather than tuning it away. (iv) Constraint handling: Constrained NSGA-II uses Deb's rules, plain NSGA-II ignores constraints (infeasible points are filtered at the end), all other population methods use the same static penalty F + 1 + 10*CV (CV normalised).

1. Problem formulations (P1–P14)

Common conventions: every objective is converted to minimisation (Max f = Min -f); each metric is computed after normalising with a fixed ideal/nadir pair per problem, $f\tilde{} = (f - \text{ideal})/(\text{nadir} - \text{ideal})$, and the hypervolume reference point is (1.1,...,1.1) in normalised space; termination is a budget of 10,000 evaluations (1,500 for P9) unless stated. IGD uses the analytic or numerically generated true front; Spacing is the standard deviation of nearest-neighbour distances.

#	Standard form min F(x) s.t.	Encoding & constraint handling	Normalisation (ideal / nadir), HV ref
P1	min ((x-3)^2, (x-7)^2), $1 \le x \le 10$. Pareto set $3 \le x \le 7$; f2=(4-√f1)^2 (convex)	1 real variable; bounds only	ideal (0,0), nadir (16,16)
P2	min (100/x, x^2), $1 \le x \le 10$ (real and integer)	real x; integer version rounds x inside the evaluation; bounds only	ideal (10,1), nadir (100,100)
P3	ZDT2, n=10: f1=x1, g=1+Σx_i, f2=g(1-(f1/g)^2), x in [0,1]^10 (non-convex)	10 reals; bounds only	ideal (0,0), nadir (1,1)
P4	min (w'Σw, -μ'w), Σw=1, $w \ge 0$ (μ, Σ as given)	4 reals in [0,1]; **repair by normalisation** w=x/Σx (default) or penalty |Σx-1|-0.01 (Section 8)	ideal (0.00704, -0.15), nadir (0.09, -0.0927); front by my own eps-constraint sweep
P5	min (7850 L b h, 4PL^3/(E b h^3)) s.t. b h^2 ≥ 2.4e-5, b in [0.01,0.1], h in [0.02,0.2]	2 reals; CV = max(0, 2.4e-5-bh^2)/2.4e-5; analytic true front h=min(0.2,100A), b=A/h	ideal (3.846 kg, 2.5e-5 m), nadir (157 kg, 0.01701 m)
P6	Binh-Korn: min (4x1^2+4x2^2, (x1-5)^2+(x2-5)^2) s.t. $(x1-5)^2+x2^2 \le 25$, (x1-8)^2+(x2+3)^2≥7.7	2 reals, CV = sum of normalised violations; true front x1=x2 (0..3) ∪ x2=3 (x1 3..5)	ideal (0,4), nadir (136,50)
P7	min (-0.8x1, -1.2x2) s.t. $x1+x2 \le 90$, $x1 \ge 10$, $x2 \ge 20$	2 reals in [0,100]; CV normalised; front is the line x1+x2=90, x1∈[10,70]	ideal (-56,-96), nadir (-8,-24); DM point (40,50) = (-32,-60)
P8	bi-objective TSP, 15 cities: min (total distance, total time), time=d*u, u~U(0.5,2) symmetric	permutation of 15 cities (random seed 7, coordinates printed in Section 8); operators OX + inversion/swap	ideal/nadir from my own 2-opt tours of each objective: ideal (351.8, 366.3), nadir (411.4, 463.7)
P9	min (1-CV accuracy of 5-NN with 5-fold CV, #features), mask≠0	30-bit mask; all-zero mask is infeasible (CV=1); real-coded methods threshold at 0.5	ideal (0,1), nadir (0.40,30); 5-NN and 5-fold CV coded by me
P10	DTLZ2, M=3 and M=5, n=M+9 variables in [0,1]	reals; bounds only; true front = unit-sphere simplex-lattice points	ideal 0, nadir 1; MC hypervolume (40,000 common random samples)
P11	max (4xA+6xB, xA+xB) s.t. $2xA+3xB \le 1200$, $4xA+2xB \le 1600$; goals profit≥2400, units≥600; priorities profit > units	LP (linprog); deviation variables d-, d+	n/a (LP methods)
P12	5 suppliers × 4 criteria, weights (0.3,0.3,0.2,0.2)	ranking only (TOPSIS, VIKOR)	vector / min-max normalisation inside the ranker
P13	(a) min (||x-a||^2, ||x-b||^2), start (0,5); (b) two-head network, classification BCE + regression MSE on synthetic data	(a) 2 reals; (b) hand-coded forward/backward pass, 10-16-1+1	loss ratios L/L(0)
P14	min (0.4+0.05x1+0.012x2, -0.2x2/(0.5+0.03x1)), x1 in [13,17], x2 in [40,90]	2 reals; bounds only; true front x1=13	ideal (1.53,-20.22), nadir (2.13,-8.99); DM point (1.8 kg, 12 h)

2. Shared toolkit (moo_utils) and unit tests

moo_utils.py contains the dominance test, Deb's constrained dominance, fast non-dominated sorting, crowding distance, rank-and-crowding survivor selection, exact 2-D hypervolume, Monte-Carlo hypervolume (M≥3), IGD, Spacing, normalisation and Das-Dennis reference points; problems.py holds P1–P14; ops.py the real (SBX, polynomial mutation), binary and permutation (order crossover, inversion/swap) operators; base.py the common initialize/evaluate/iterate/run skeleton with budget counting and the archive classes. tests/test_all.py contains 3 toolkit tests and one key-rule test for each of the 37 algorithms; all must pass before any experiment is run. Verification of the three deep-dive algorithms against the hand traces is shown in Section 5.

Test	Result
utils: car Pareto-front example	**PASS**
utils: Deb constrained-dominance table	**PASS**
utils: crowding distance, HV, IGD, spacing	**PASS**
01 Weighted Sum: convex w=(.5,.5)->x=5; never the middle of the non-convex ZDT2 front	**PASS**
02 eps-Constraint: f2<=eps holds and the point is non-dominated	**PASS**
03 Benson: f + l = z, l >= 0, solution non-dominated	**PASS**
04 Weight metric: p=inf reaches the ZDT2 middle, p=1 does not	**PASS**
05 Value function: result maximises the DM utility among random portfolios	**PASS**
06 Goal programming: P11 units goal leaves shortfall 100, profit goal met	**PASS**
07 Lexicographic: profit first (2400, tie) then units -> (300,200)	**PASS**
08 TOPSIS: dominant alternative has closeness 1, anti-ideal has 0	**PASS**
09 VIKOR: dominant alternative has S=R=Q=0	**PASS**
10 VEGA: sub-population m is selected on objective m only	**PASS**
11 MOGA: rank = 1 + number of dominators	**PASS**
12 NSGA: every member of front k has higher dummy fitness than every member of front k+1	**PASS**
13 NPGA: candidate dominated by the comparison set loses	**PASS**
14 NSGA-II: elitism - hypervolume of the non-dominated set never decreases; boundary crowding = inf	**PASS**
15 NSGA-III: association = nearest reference line; intercept normalisation maps extremes to 1	**PASS**
16 SPEA2: raw fitness 0 <=> non-dominated; archive size stays fixed	**PASS**
17 PESA-II: archive non-dominated; region tournament favours the less crowded box	**PASS**
18 MOEA/D: neighbourhoods are the T nearest weight vectors; Tchebycheff g	**PASS**
19 MOEA/D-DE: trial = x_r1 + F (x_r2 - x_r3) when CR=1 and mutation off	**PASS**
20 MOEA/D-ACO: constructed tours are permutations; deposit raises pheromone on used edges	**PASS**
21 Constrained NSGA-II: Deb's rules rank feasible above infeasible	**PASS**
22 MOPSO: velocity rule v' = w v + c1 r1 (pbest-x) + c2 r2 (leader-x); archive non-dominated	**PASS**
23 MOGWO: alpha/beta/delta are archive members; wolves stay inside the bounds	**PASS**
24 MOABC: greedy selection by dominance (+1 / 0 / -1)	**PASS**
25 MOACO: transition probabilities sum to 1 and exclude visited cities	**PASS**
26 MOSA: dominating move accepted (p=1); worse move accepted with exp(-w.dF/T)	**PASS**
27 MODE: trial replaces target only if it dominates	**PASS**
28 MO-TLBO: teaching factor shrinks monotonically from 2 to 1	**PASS**
29 MOBA: loudness decreases, pulse rate increases	**PASS**
30 MOMA: hill-climbing local search never worsens the scalarised objective	**PASS**
31 RVEA: every solution is assigned to the reference vector with the smallest angle	**PASS**
32 IBEA: a dominated point has lower fitness; selection keeps exactly N	**PASS**
33 SMS-EMOA: hypervolume contributions identify the point to discard	**PASS**
34 HypE: Monte-Carlo k=1 fitness approximates the exact exclusive contribution	**PASS**
35 R-NSGA-II: the selected set is closer to the reference point than the rest	**PASS**
36 MGDA: <d*,g_i> >= ||d*||^2 for all tasks; opposite equal gradients give d*=0	**PASS**
37 GradNorm: weights stay positive and sum to T; the slower-learning task gets more weight	**PASS**

40 of 40 tests pass.

3. Task A — implementation log

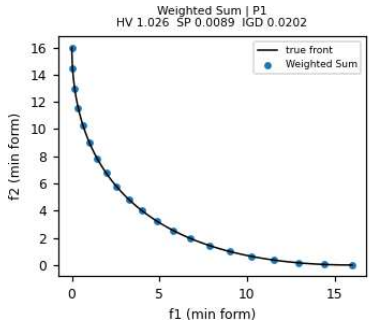
Seed 1, budget 10,000 evaluations (P9: 1,500). HV and Spacing are computed on the normalised front; IGD only where the true front is known. For algorithms 5–9 and 36–37 the output is a solution or ranking, so HV/IGD are not meaningful (n/a); their results are in the figures and in Task D. Statuses: 'works' means the key-rule unit test passes and the run is plotted; remarks mark behaviour a reader should know about.

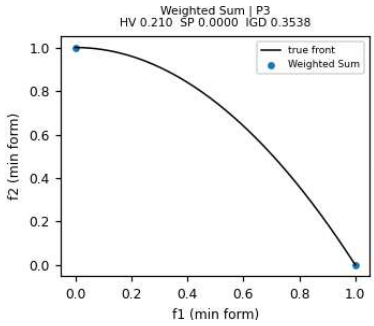
#	Algorithm	Problems (HV / IGD / Spacing, seed 1)	Budget	Runtime s	Status
1	Weighted Sum	P1: HV 1.026, IGD 0.0202, SP 0.0089 (n=21) P3: HV 0.210, IGD 0.3538, SP 0.0000 (n=2) P4: HV 0.938, IGD 0.0262, SP 0.0618 (n=17)	closed form / sweep	1.5	works
2	eps-Constraint	P2: HV 1.086, IGD 0.0371, SP 0.1291 (n=21) P3: HV 0.514, IGD 0.0389, SP 0.0605 (n=19) P5: HV 1.150, IGD 0.2318, SP 0.1935 (n=21)	closed form / sweep	2.2	works
3	Benson's Method	P1: HV 0.994, IGD 0.0704, SP 0.0923 (n=15) P7: HV 0.673, IGD 0.0246, SP 0.0096 (n=15)	closed form / sweep	1.3	works
4	Weight Metric (p-norm, Tchebycheff)	P1: HV 1.022, IGD 0.0434, SP 0.0335 (n=21) P3: HV 0.517, IGD 0.0180, SP 0.0108 (n=21)	closed form / sweep	1.5	works
5	Value Function	P4; P14: solution(s) / ranking, see Fig. A-05	closed form / sweep	-	works
6	Goal Programming	P11; P7: solution(s) / ranking, see Fig. A-06	closed form / sweep	-	works
7	Lexicographic	P11; P7: solution(s) / ranking, see Fig. A-07	closed form / sweep	-	works
8	TOPSIS	P12; P4: solution(s) / ranking, see Fig. A-08	closed form / sweep	-	works
9	VIKOR	P12; P4: solution(s) / ranking, see Fig. A-09	closed form / sweep	-	works
10	VEGA	P2: HV 1.112, IGD 0.0136, SP 0.0375 (n=100) P3: HV 0.000, IGD 1.0187, SP 0.0000 (n=1)	10000	0.0	works - collapses on ZDT2 (by design: sub-populations favour extremes)
11	MOGA	P3: HV 0.439, IGD 0.0560, SP 0.0203 (n=44) P5: HV 1.193, IGD 0.0124, SP 0.0250 (n=62)	10000	0.2	works
12	NSGA	P5: HV 1.190, IGD 0.0132, SP 0.0159 (n=57) P7: HV 0.636, IGD 0.0370, SP 0.0249 (n=26)	10000	0.2	works
13	NPGA	P3: HV 0.035, IGD 0.4767, SP 0.0421 (n=11) P5: HV 1.190, IGD 0.0180, SP 0.0202 (n=53)	10000	1.9	works - weak on ZDT2
14	NSGA-II	P3: HV 0.510, IGD 0.0196, SP 0.0065 (n=100) P5: HV 1.190, IGD 0.0079, SP 0.0514 (n=54) P6: HV 1.020, IGD 0.0051, SP 0.0058 (n=98)	10000	0.9	works
15	NSGA-III	P10_3: HV 0.561, IGD 0.1204, SP 0.0642 (n=88) P10_5: HV 0.492, IGD 0.4621, SP 0.1001 (n=69)	9936	0.8	works - under-converged at 10k evals on DTLZ2 (30k: IGD 0.055, best of 3 methods)
16	SPEA2	P4: HV 0.968, IGD 0.0044, SP 0.0024 (n=100) P5: HV 1.195, IGD 0.0051, SP 0.0028 (n=100)	10000	6.3	works
17	PESA-II	P3: HV 0.528, IGD 0.0092, SP 0.0081 (n=100) P5: HV 1.194, IGD 0.0089, SP 0.0094 (n=100)	10000	2.0	works
18	MOEA/D	P3: HV 0.523, IGD 0.0092, SP 0.0070 (n=88) P10_3: HV 0.665, IGD 0.0865, SP 0.0595 (n=64)	10000	2.6	works
19	MOEA/D-DE	P3: HV 0.535, IGD 0.0049, SP 0.0033 (n=97) P5: HV 1.193, IGD 0.1230, SP 0.0828 (n=100)	10000	2.2	works
20	MOEA/D-ACO	P8: HV 0.740, IGD n/a, SP 0.1322 (n=8)	10000	3.0	works
21	Constrained NSGA-II	P5: HV 1.195, IGD 0.0067, SP 0.0076 (n=97) P6: HV 1.020, IGD 0.0053, SP 0.0056 (n=97) P7: HV 0.696, IGD 0.0066, SP 0.0057 (n=82)	10000	1.1	works
22	MOPSO	P4: HV 0.966, IGD 0.0070, SP 0.0063 (n=100) P5: HV 1.195, IGD 0.0073, SP 0.0081 (n=100)	10000	0.8	works
23	MOGWO	P5: HV 1.195, IGD 0.0059, SP 0.0142 (n=100) P9: HV 1.128, IGD n/a, SP 0.0377 (n=6)	10000 (P9: 1500)	6.0	works - weak on ZDT2; needed bound re-sampling (clipping collapsed the archive)
24	MOABC	P3: HV 0.527, IGD 0.0103, SP 0.0143 (n=92) P6: HV 1.021, IGD 0.0044, SP 0.0030 (n=100)	9950	0.9	works
25	MOACO	P8: HV 0.757, IGD n/a, SP 0.1012 (n=10)	10000	2.5	works
26	MOSA	P3: HV 0.388, IGD 0.1014, SP 0.0674 (n=22) P5: HV 1.195, IGD 0.0051, SP 0.0037 (n=100)	10000	0.7	works - weak on ZDT2
27	MODE	P3: HV 0.110, IGD 0.6098, SP 0.0000 (n=1) P5: HV 1.195, IGD 0.0048, SP 0.0054 (n=100)	10000	0.6	works - collapses on ZDT2 under the strict dominance-replacement rule
28	MO-TLBO	P3: HV 0.281, IGD 0.2282, SP 0.0233 (n=100) P5: HV 1.195, IGD 0.0264, SP 0.0206 (n=100)	9900	0.7	works - weak on ZDT2
29	MOBA	P3: HV 0.514, IGD 0.0159, SP 0.0055 (n=100) P4: HV 0.961, IGD 0.0139, SP 0.0052 (n=100)	10000	0.6	works
30	MOMA	P5: HV 1.195, IGD 0.0068, SP 0.0068 (n=98) P9: HV 1.069, IGD n/a, SP 0.0130 (n=5)	9836 (P9: 1500)	5.5	works
31	RVEA	P10_3: HV 0.497, IGD 0.1493, SP 0.0357 (n=56)	9919	0.1	works - only 56 points on M=3 (empty vectors discard)
32	IBEA	P3: HV 0.522, IGD 0.0118, SP 0.0088 (n=75) P14: HV 0.704, IGD 0.0042, SP 0.0036 (n=92)	10000	0.7	works

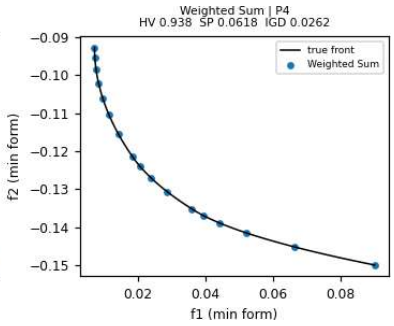
#	Algorithm	Problems (HV / IGD / Spacing, seed 1)	Budget	Runtime s	Status
33	SMS-EMOA	P3: HV 0.511, IGD 0.0220, SP 0.0089 (n=100) P14: HV 0.705, IGD 0.0035, SP 0.0008 (n=100)	10000	16.0	works
34	HypE	P10_3: HV 0.696, IGD 0.1290, SP 0.0235 (n=91)	9919	24.8	works - slow (Monte-Carlo contributions)
35	R-NSGA-II	P5: HV 1.194, IGD 0.0123, SP 0.0187 (n=83) P14: HV 0.698, IGD 0.0051, SP 0.0034 (n=100)	10000	2.0	works
36	MGDA	P13: solution(s) / ranking, see Fig. A-36	closed form / sweep	-	works
37	GradNorm	P13: solution(s) / ranking, see Fig. A-37	closed form / sweep	-	works

Per-algorithm figures and results (Fig. A-01 … A-37)

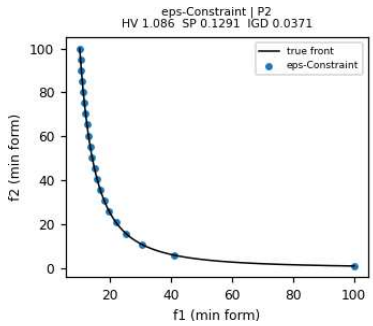
1. Weighted Sum

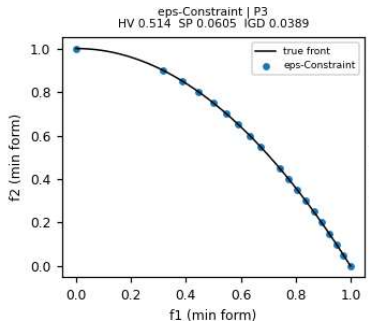


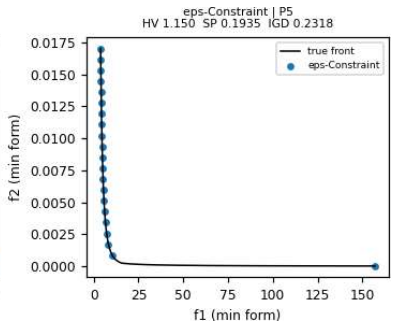




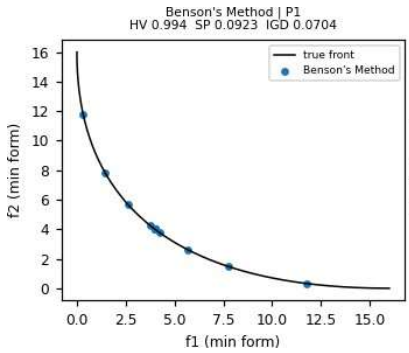
2. eps-Constraint

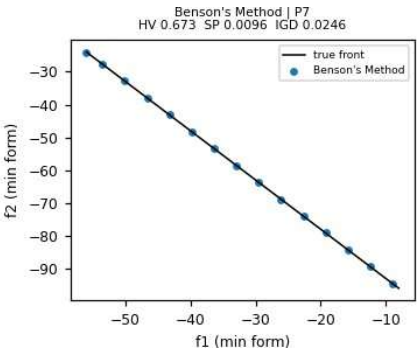




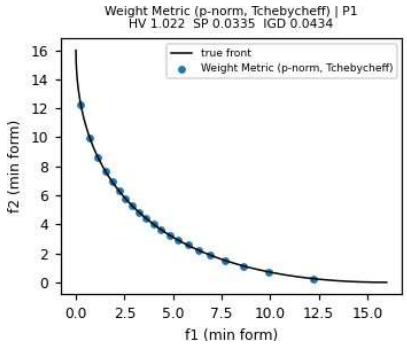


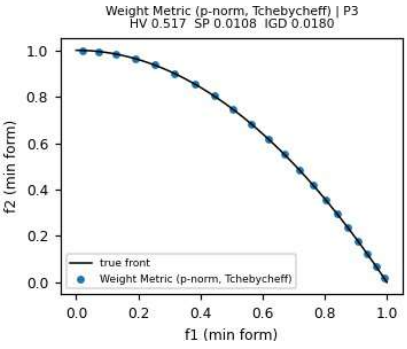
3. Benson's Method



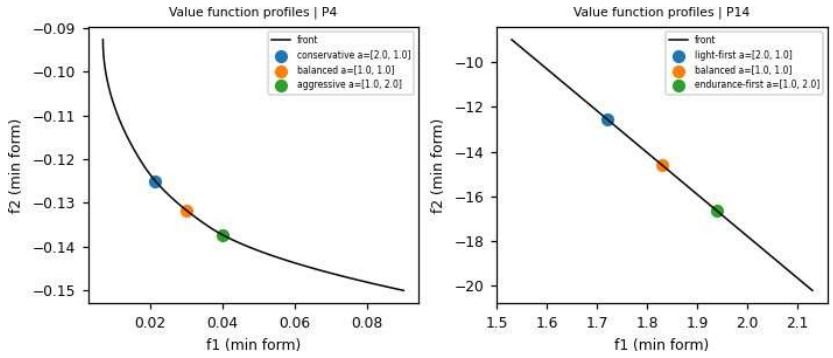


4. Weight Metric (p-norm, Tchebycheff)



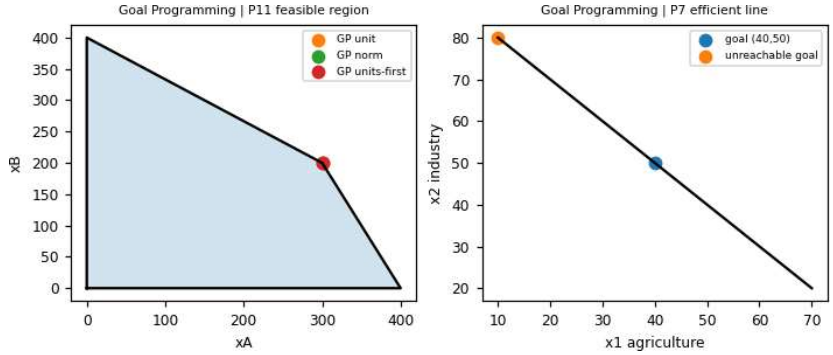


5. Value Function



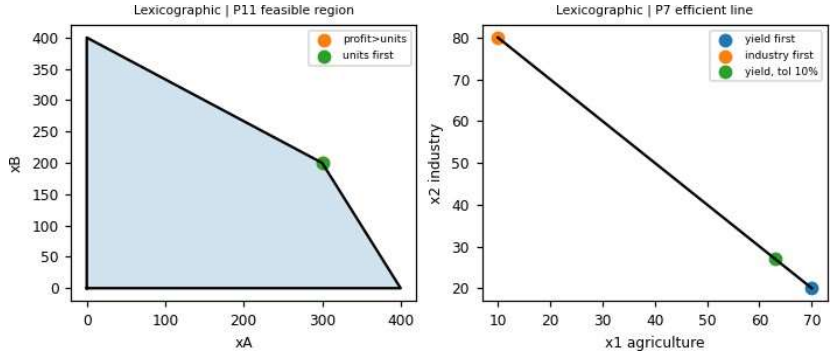
Value function: profiles P4 conservative / balanced / aggressive map to weights (0.0,0.395,0.342,0.263) … see taskA.json; P14 profiles give 1.72 kg / 12.5 h, 1.83 kg / ... (JSON).

6. Goal Programming



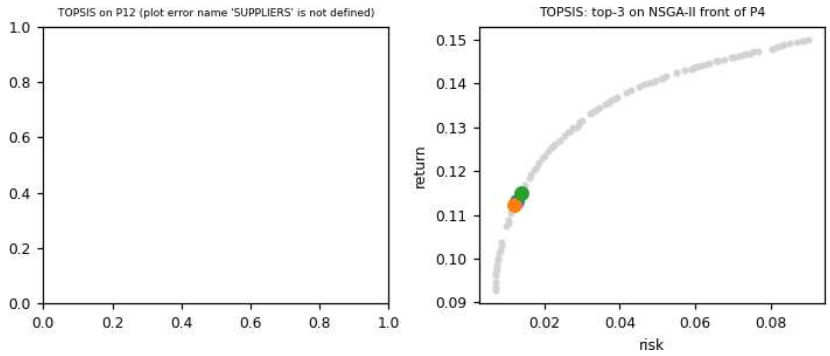
P11: unit-weight and normalised-weight goal programming both give x = (300,200), profit goal met, units shortfall exactly 100 (the unavoidable minimum). P7: goal (32,60) i.e. (40,50) is reached exactly with zero deviation.

7. Lexicographic



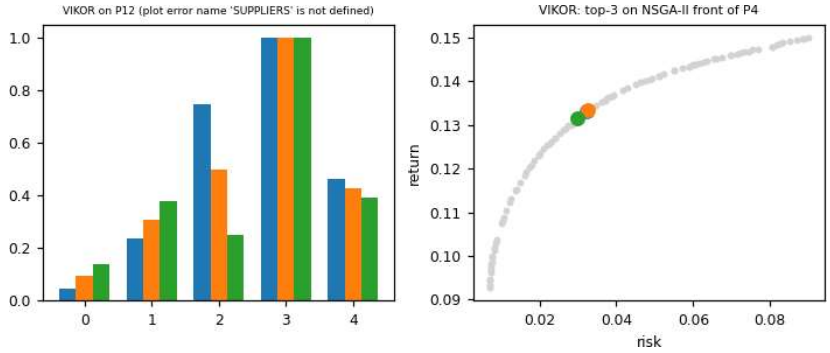
P11: profit-first gives 2400 (tie along (300,200)-(0,400)), the second priority picks (300,200) with 500 units. Units-first gives the same x; P7: yield-first (70,20), industry-first (10,80).

8. TOPSIS



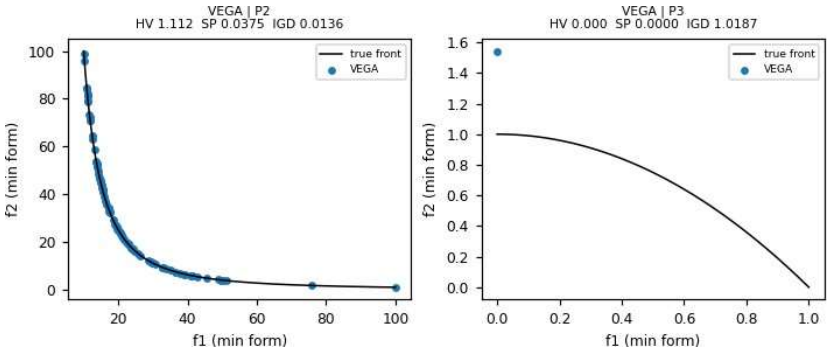
P12: Pareto-optimal suppliers = ['S1', 'S2', 'S3', 'S4', 'S5']. TOPSIS ranks ['S3', 'S1', 'S5', 'S2', 'S4']; after shifting 0.1 of weight from cost to quality ['S3', 'S5', 'S1', 'S2', 'S4'] (the order of S1 and S5 flips).

9. VIKOR

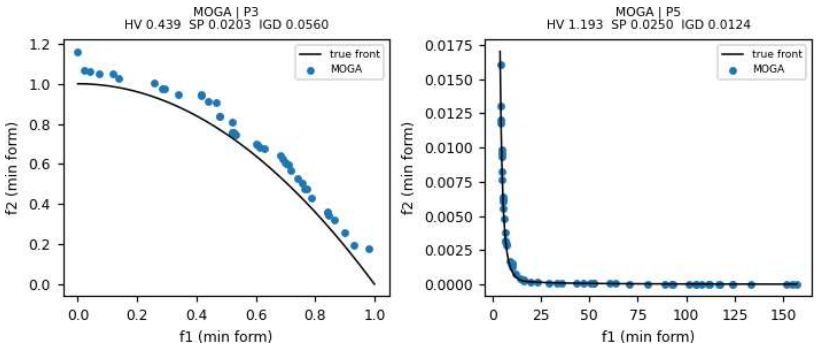


VIKOR ranks (v=0.25 / 0.5 / 0.75): ['S1', 'S2', 'S5', 'S3', 'S4'] / ['S1', 'S2', 'S5', 'S3', 'S4'] / ['S1', 'S3', 'S2', 'S5', 'S4']; after the weight shift: ['S1', 'S3', 'S5', 'S2', 'S4'].

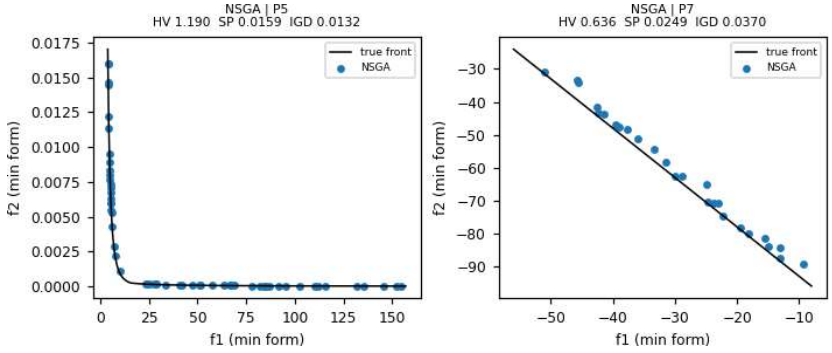
10. VEGA



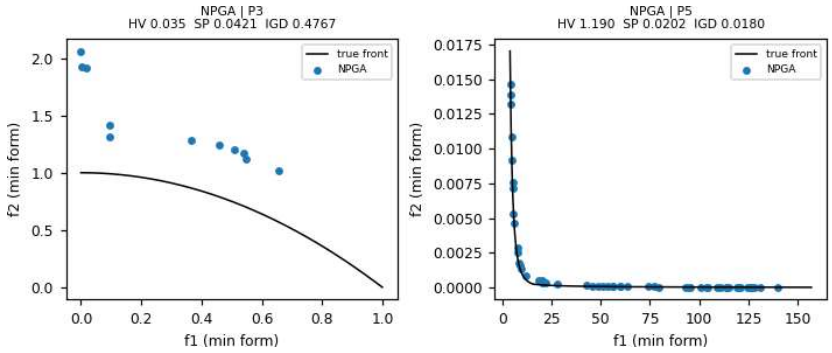
11. MOGA



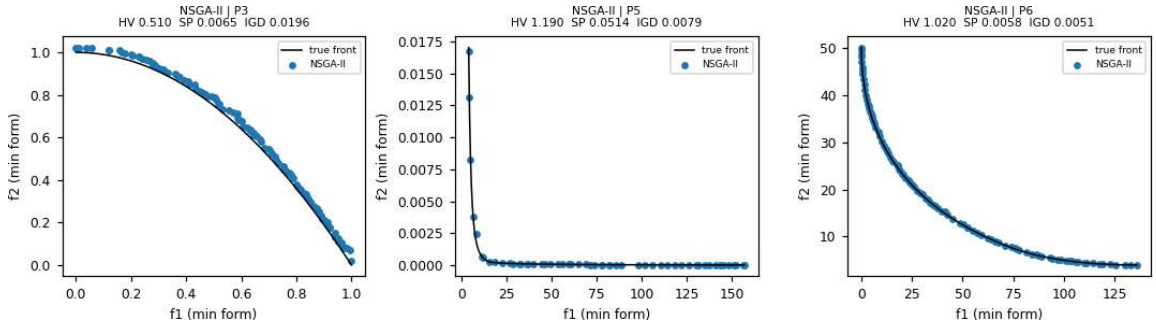
12. NSGA



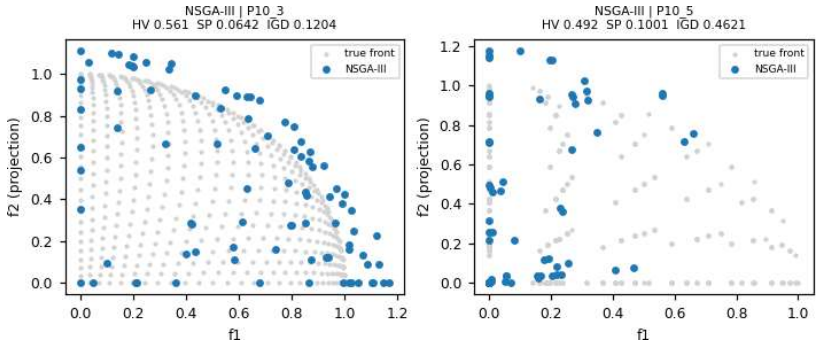
13. NPGA



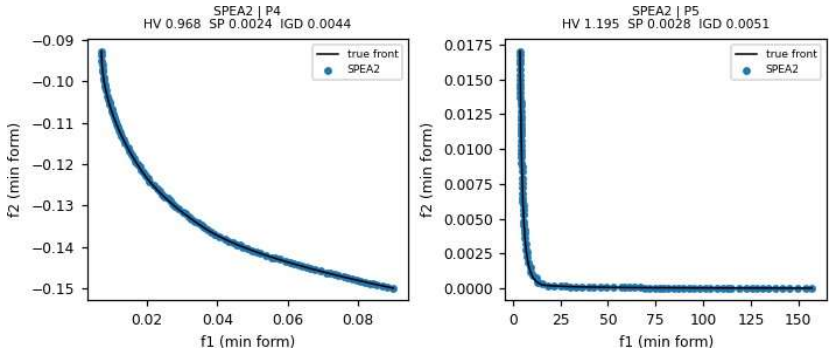
14. NSGA-II



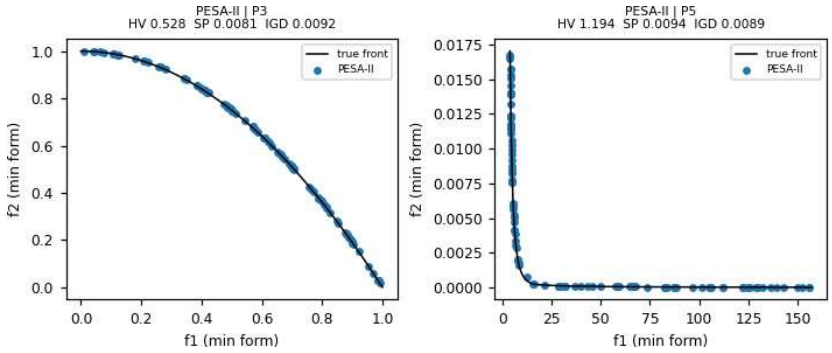
15. NSGA-III



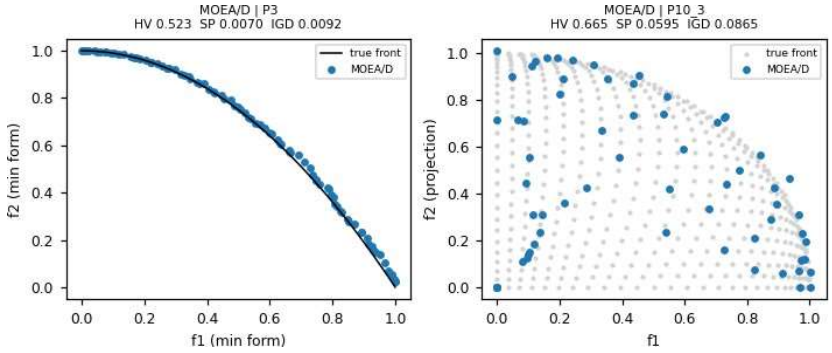
16. SPEA2



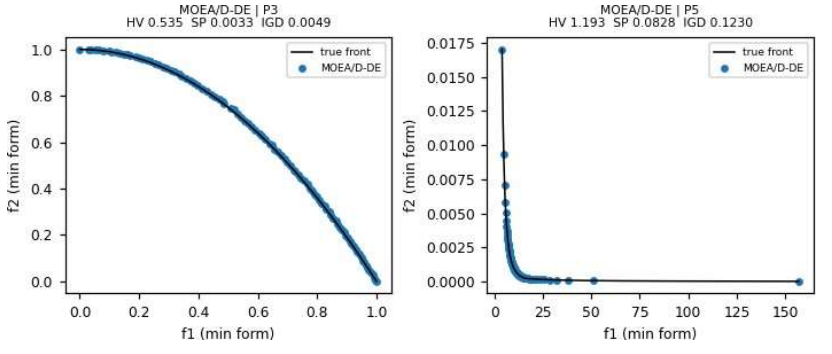
17. PESA-II



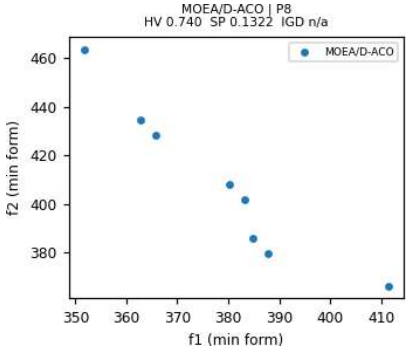
18. MOEA/D



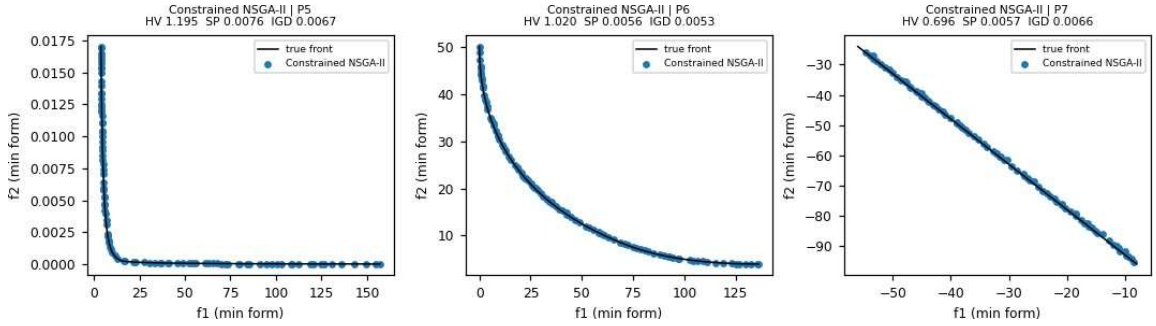
19. MOEA/D-DE



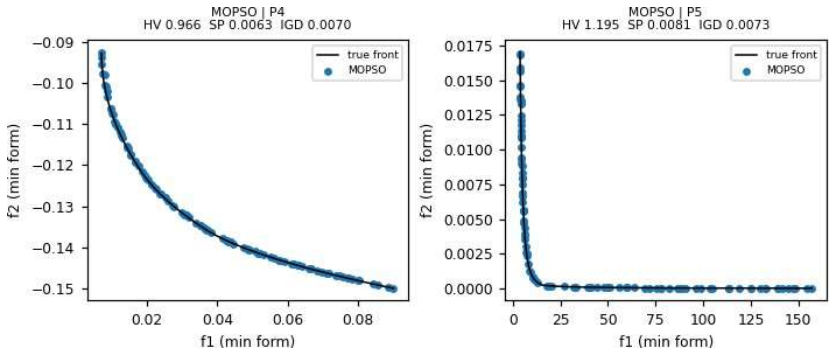
20. MOEA/D-ACO



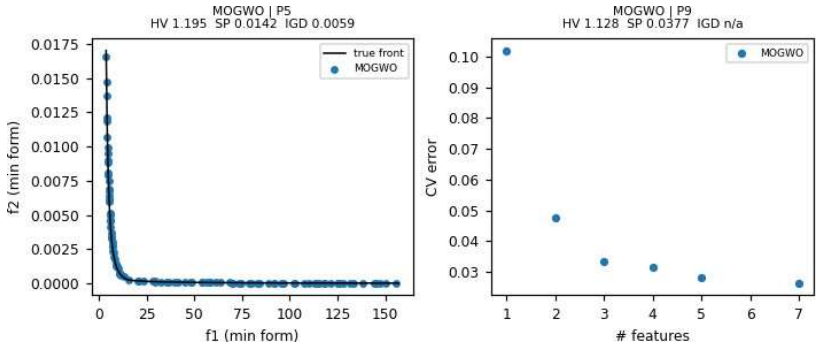
21. Constrained NSGA-II



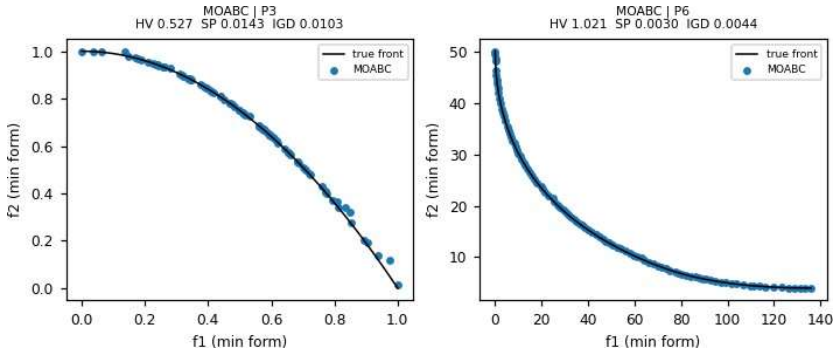
22. MOPSO



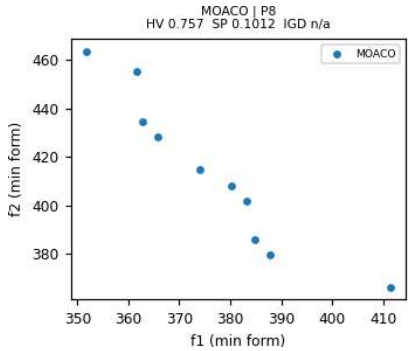
23. MOGWO



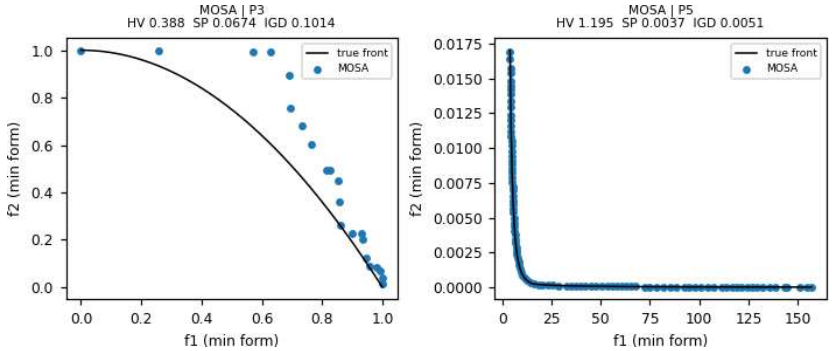
24. MOABC



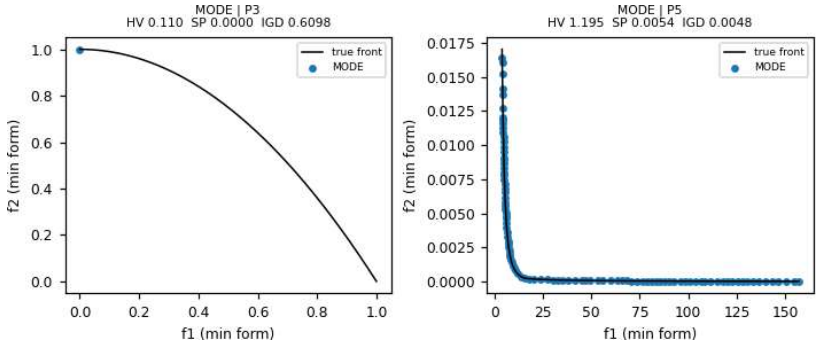
25. MOACO



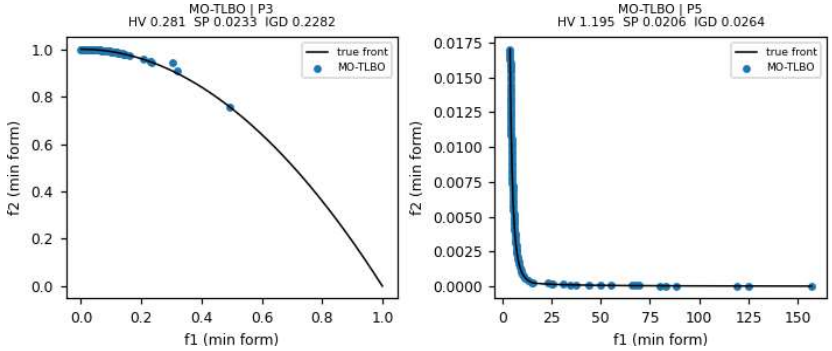
26. MOSA



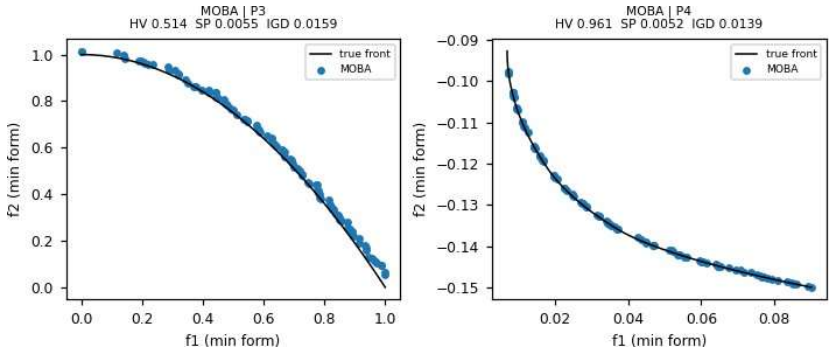
27. MODE



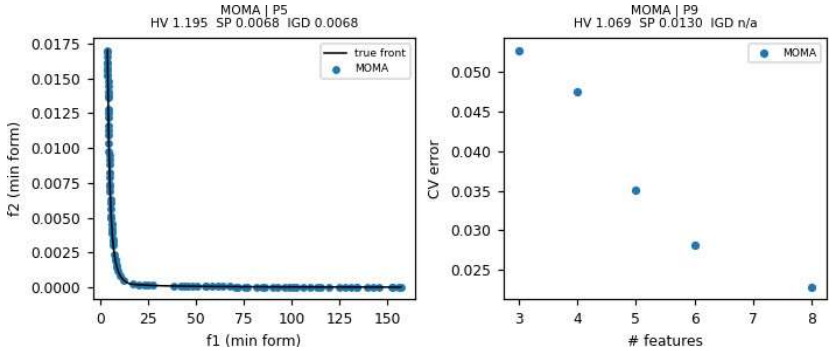
28. MO-TLBO



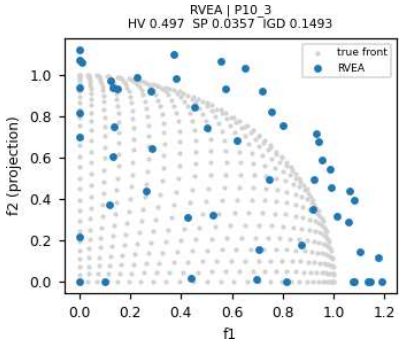
29. MOBA



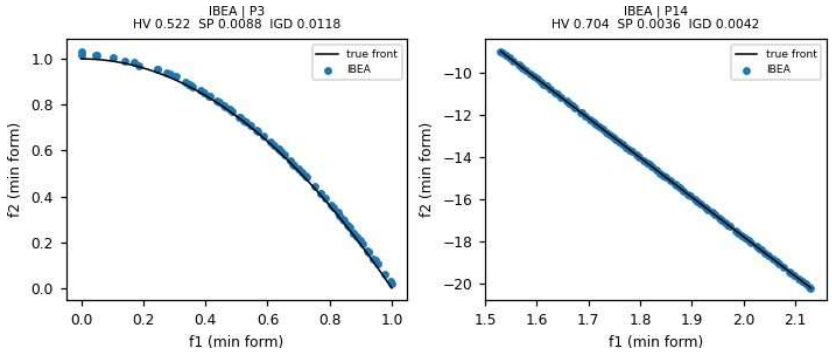
30. MOMA



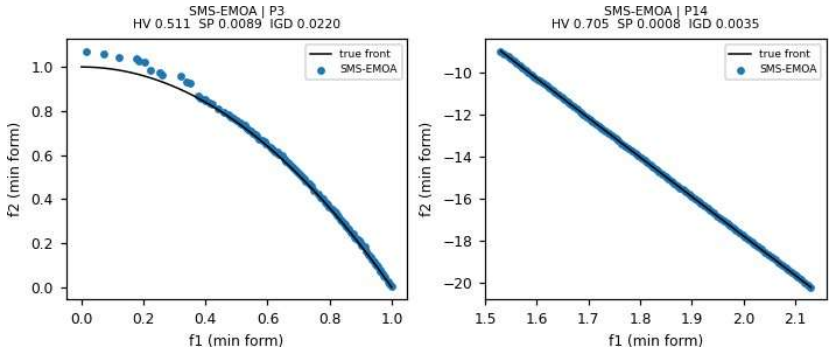
31. RVEA



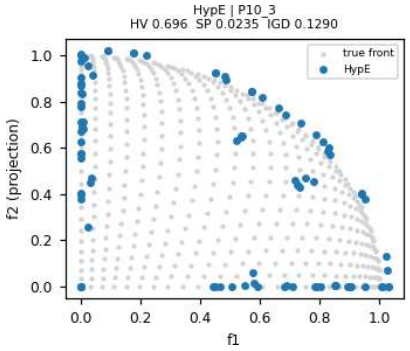
32. IBEA



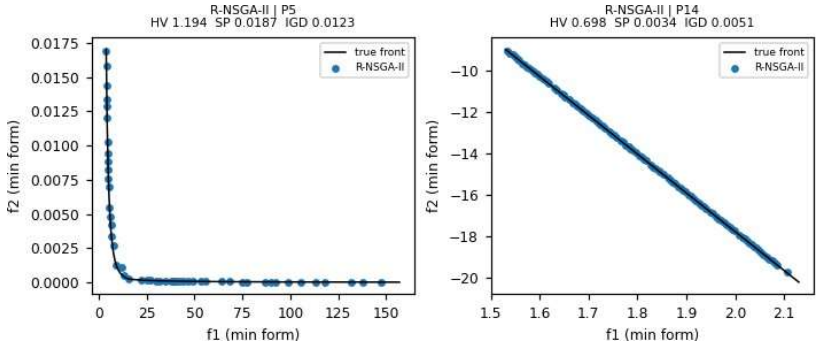
33. SMS-EMOA



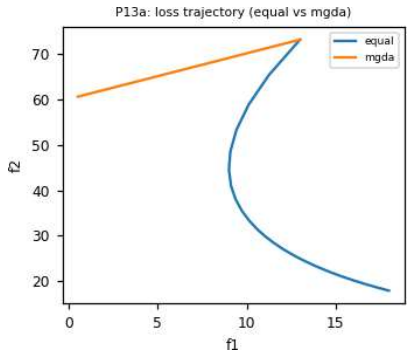
34. HypE



35. R-NSGA-II

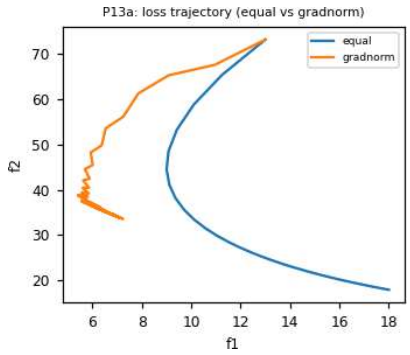


36. MGDA



P13a: equal weights end at (0,0) with losses (18,18); MGDA ends at (2.5,2.5), the projection of the start (0,5) on the Pareto segment, with losses (0.5,60.5). Network results are in Section 6 (Task D).

37. GradNorm



P13a: GradNorm weights end at (1.63,0.37); network results are in Section 6 (Task D).

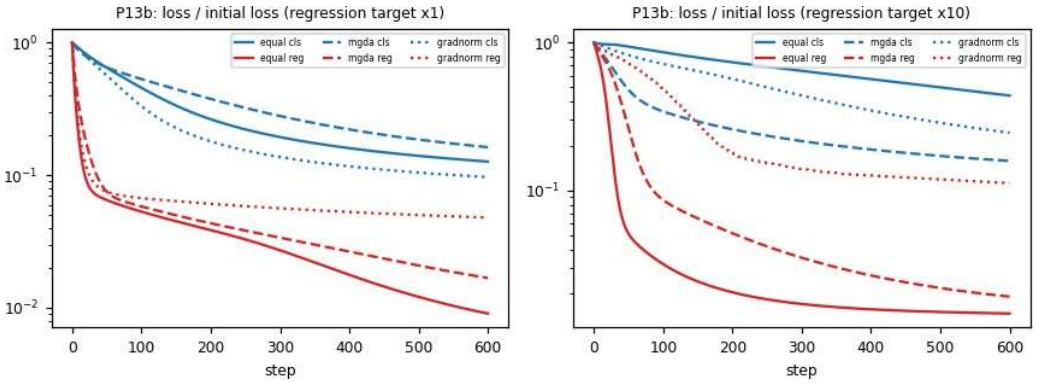


Fig. A-P13b: P13(b) network, loss divided by initial loss for equal weights, MGDA and GradNorm, regression target scale 1 (left) and 10 (right).

4. Task B — master comparison table (all 37 algorithms)

Rows are grouped by family. Every algorithm lists at least four advantages and four disadvantages; at least two of each cite a number from my own runs (Task A seed 1, or Task D 10 seeds, or Task C/extras) with the figure; the other points are justified by the algorithm's steps. 'Best' and 'avoid' state the situation in which the algorithm should or should not be chosen.

Family: Classical

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
1	**Weighted Sum**	Scalarise F by sum_i w_i f~_i, solve the single-objective problem (SLSQP, 6 random starts) and repeat for 21 weights in [0,1].	• Cheapest scheme: one smooth single-objective solve per point (about 0.3-0.7 s for the whole 21-weight sweep on P1/P4, Fig. A-01). • On the convex P1 front all 21 weights give 21 different Pareto points, IGD 0.020 (Fig. A-01). • Any gradient solver can be reused; constraints (equality on P4) are passed straight to the solver; scales to many variables. • The only parameter, the weight, has an intuitive meaning (relative importance); the obtained points are provably Pareto-optimal on convex fronts.	• Cannot reach a non-convex front: only 2 points on ZDT2, HV 0.210 against 0.542 for the true front (Fig. A-01). • Equal weight steps do not give equal spacing: nearest-point steps on P1 range from 0.071 to 0.098 (normalised units); on P4 Spacing is 0.062, the worst of the P4 methods. • Each solve gives one point and gradient solvers can stop in local optima on multimodal problems. • Needs normalised objectives and a sensible weight grid; a priori weights hide the shape of the trade-off.	Convex, smooth, cheap-to-evaluate problems with a gradient solver at hand (P1, P4).	Non-convex or disconnected fronts (ZDT2/P3), discrete or noisy objectives.
2	**eps-Constraint**	Minimise one objective f1 and turn the others into constraints f2 <= eps; sweep 21 eps values between the ideal and nadir values.	• Reaches non-convex fronts: HV 0.514 on P3, where weighted sum finds 2 points (Fig. A-02). • Handles the constrained P5 and returns 21 feasible points covering the whole range of the front (Fig. A-02). • eps has a direct physical meaning (an acceptable deflection, delay or cost bound). • Every solution is Pareto-optimal provided the constraint is active, and the front can be sampled as densely as wished.	• Equal eps steps give uneven points: Spacing 0.194 and IGD 0.232 on P5, 0.129 on P2 (Fig. A-02). • Sub-problems with a tight eps can be infeasible; the nadir must be known to choose the eps range. • Needs a constrained solver (SLSQP with multi-start here); cost grows as (points)^(M-1) for M objectives. • A bad eps range wastes solves on dominated/infeasible settings.	Bi-objective problems where one objective has a natural bound (budget, deflection limit); non-convex fronts.	Many objectives (grid of eps explodes); problems whose feasibility for a given eps is hard to find.
3	**Benson's Method**	For a reference point z that some feasible point improves on, maximise sum w_i l_i subject to f_i(x) + l_i = z_i, l_i >= 0.	• Result is Pareto-optimal and dominates z by construction: l = z - f(x), checked in the hand trace of Task C (x = 5 and x = 4.414). • With a tight but attainable z it reaches the dented middle of ZDT2: 5 mid-front points at offset 0.2 (Fig. C-1, Task C). • z is an intuitive input (a current design to be improved); also usable as a Pareto-optimality test of any given point. • Provides the improvement vector l, a measure of how much each objective can be bettered.	• Needs an attainable z: for z = (t, 1-t)+0.02 the sub-problem was infeasible for mid-front t on ZDT2, leaving only 2 points (Fig. C-1). • If z lies beyond the nadir the constraints are inactive and it reduces to weighted sum (extreme points only for offset 0.6, Fig. C-1). • On P7 (linear front) equal weights give a degenerate tie along the front; unequal weights were needed: HV 0.673, Spacing 0.0096. • Extra l variables and equality constraints per solve; sweep of z and w required to build a front.	Improving an existing design or screening a candidate for Pareto optimality.	When no feasible reference point is known or for large objective counts.
4	**Weight Metric (p-norm, Tchebycheff)**	Minimise (sum w_i |f~_i|^p)^(1/p) (distance to the ideal point) for p = 1, 2 and infinity (Tchebycheff via an epigraph variable t).	• p = infinity reaches the concave front: HV 0.517, IGD 0.018 on ZDT2, where p = 1 finds only end points (Fig. A-04). • On P1 the p = 2 and p = infinity metrics return 21 distinct Pareto points; p = 1 reproduces weighted sum (21 distinct). • Unified family: p controls the convexity assumption; Tchebycheff solutions are weakly Pareto optimal for every front shape. • Weights keep their meaning of relative importance.	• Spacing is uneven: Spacing for p = 1, 2, inf on P1 is 0.0080 / 0.0300 / 0.0335, so equal weight steps cluster points near the middle. • The Tchebycheff non-smooth max needs an epigraph reformulation with extra variable and constraints. • Requires the ideal point (a pre-solve of every objective) and normalisation. • Only weak Pareto-optimality is guaranteed for p = infinity; one solve per point.	Bi-objective problems where the front may be non-convex and a gradient solver is available.	Black-box problems without a solver or with expensive solves per weight.
5	**Value Function**	Maximise the decision maker's utility V(f~) = prod (1.05 - f~_i)^a_i; different exponents a model different profiles.	• Returns one preferred solution per profile (3 investor profiles on P4, 3 laptop profiles on P14, Fig. A-05) - no front needed. • Solutions lie on the Pareto front (all profile points sit on the plotted front, Fig. A-05). • Encodes trade-off attitude (risk-averse vs return-seeking) directly, so the analyst gets an answer, not a menu. • Cheap: one single-objective solve per profile.	• Needs a utility function; a wrong V gives a confidently wrong answer (a=(2,1) vs (1,2) moves P4 from the safe to the risky end). • Does not show the alternatives, so the decision maker cannot learn the trade-off. • Utility elicitation is hard and non-unique; exponents are not comparable across people. • Non-convex utilities may need multi-start (6 starts used).	A single decision maker whose preferences are known (investor profiles on P4, laptop on P14).	Exploratory design where preferences are unknown or several stakeholders disagree.
6	**Goal Programming**	Targets T_i with deviation variables d-, d+: f_i + d-_i - d+_i = T_i, minimise weighted unwanted deviations (LP).	• Handles unreachable targets gracefully: P11 units goal leaves exactly the minimum shortfall 100 at x = (300, 200), profit goal met (Fig. A-06). • Unreachable targets still give a sensible compromise: on P7 an unreachable goal (60,110) returns (10,80) with deviations (52,14), the minimum total (Fig. A-06). • For linear problems it is an LP: exact, fast, no tuning. • Targets are natural for managers (profit >= 2400, units >= 600).	• Answer depends entirely on the targets: on P7 a reachable goal (32, 60) is hit exactly (40, 50) while an unreachable one gives a different compromise. • Returns a single compromise, not a front; weights decide which goal is sacrificed. • On P11 unit weights, 1/target weights and a 10x priority on units all return the same x = (300,200) (only the objective value changes: 100, 0.167, 1000), so the solution is insensitive but the objective value is not comparable between weightings. • Poorly set goals can return a dominated solution if only one-sided deviations are penalised wrongly.	Linear/convex planning with explicit targets (P11, P7).	When targets are unknown or the real aim is the full trade-off curve.

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
7	**Lexicographic**	Solve objective 1, fix it (within a tolerance), solve objective 2 under that restriction, and so on.	• P11: profit first gives 2400 on a whole edge, second priority then selects (300, 200) with 500 units (Fig. A-07). • Priority order changes the answer clearly on P7: yield first gives (70, 20), industry first gives (10, 80) (Fig. A-07). • No weights needed, only a priority order; LP solves are exact. • Transparent and easy to explain to managers.	• Strict priority means lower objectives matter only for ties: the second priority is irrelevant unless profit has ties, as in P11. • Sensitive to the tolerance: on P7 a 10% tolerance on yield moves the answer from (70, 20) to (63, 27); on P11 tol 0 and 5% gave the same point. • Gives one solution, no trade-off information. • Needs a total order of objectives, which may not exist.	Clear priorities and ties in the first objective (P11).	Objectives of similar importance.

Family: Ranking / MCDM

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
8	**TOPSIS**	Vector-normalise the matrix, weight, compute distance to the ideal and anti-ideal point, rank by C = D-/(D+ + D-).	• Ranks the 5 suppliers in one pass without optimisation: S3 > S1 > S5 > S2 > S4 (Fig. A-08). • Applied to the NSGA-II front of P4 it names the top three portfolios instantly (JSON). • No parameters except the weights; handles benefit and cost criteria. • Zero cost - O(m n).	• A 0.1 weight shift between cost and quality swaps S1 and S5 (rank flip, Task A/P12). • Compensatory: in a toy case (10,10,10,1) vs (6,6,6,6) TOPSIS gives 0.577 vs 0.598, so one terrible criterion is only mildly punished (Task D4). • Rank reversal can occur when alternatives are added or removed. • It is a ranker, not an optimiser: it needs the candidate set.	Choosing among a short list of alternatives (P12, picking from a front).	When one criterion must not be allowed to be very bad.
9	**VIKOR**	Compute S (weighted sum of regrets), R (max regret) and Q = v*S~ + (1-v)*R~; rank by Q, check acceptable advantage.	• Parameter v moves between group utility and individual regret: ranks S1,S2,S5,S3,S4 for v = 0.25/0.5 and S1,S3,S2,S5,S4 for v = 0.75 (Fig. A-09). • Protects against one very bad criterion: in the toy case (10,10,10,1) vs (6,6,6,6) VIKOR with v = 0 gives Q = 1 vs 0, i.e. the balanced alternative wins decisively, while v = 1 reverses it (Task D4). • Acceptable-advantage test says if the winner is clearly better. • Cheap and transparent.	• The ranking changes with v (S3 moves from 4th to 2nd), so v is a hidden decision. • The advantage test often fails on small data sets, then only a compromise set is returned. • Min-max normalisation is sensitive to outliers. • Not an optimiser.	Compromise decisions where a single bad criterion is unacceptable.	Large candidate sets with many near ties.

Family: Non-elitist EA

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
10	**VEGA**	Population split into M sub-populations of size N/M, sub-population m selected only on objective m; shuffle, SBX, mutation.	• Extremely cheap: the 10,000-evaluation run takes 0.0 s (Fig. A-10). • Needs no dominance computation and no diversity parameter. • Finds extreme (single-objective-best) solutions quickly; on the convex-like P2 it kept 100 points, HV 1.112. • Very simple to implement.	• Collapsed to 1 point on ZDT2, HV 0.000: speciation towards the extremes (Fig. A-10, A-HV history E-2). • Misses middle trade-offs: sub-populations favour extremes ('speciation'). • No elitism: the final front is only the last population. • No diversity mechanism, so spacing is poor (0.038 on P2).	Quick demonstration or when only the extremes matter.	Concave fronts or any task needing evenly spread compromises.
11	**MOGA**	Rank = 1 + number of dominators, linear rank-to-fitness map, fitness sharing (sigma 0.1) inside each rank, roulette selection.	• Pareto-based: finds interior points (HV 1.193 on P5, Fig. A-11). • Sharing keeps a spread: 44 distinct points on ZDT2. • Rank definition is simple; no gradient needed. • Works with any encoding.	• Poorer than elitist methods on ZDT2: HV 0.439 and IGD 0.056 vs NSGA-II 0.510 / 0.020 (Fig. A-11). • No elitism: good solutions can be lost between generations. • sigma_share must be tuned; sharing costs O(N^2) per rank. • Rank-based selection pressure is weak early on.	Historical baseline, small low-dimensional problems.	Production use; prefer NSGA-II/SPEA2.
12	**NSGA**	Non-dominated sorting into fronts, dummy fitness reduced by niche sharing within each front, roulette selection.	• Front-wise dummy fitness gives good convergence on P5: HV 1.190, IGD 0.013 (Fig. A-12). • Sharing inside a front keeps diversity: 57 points. • Pure dominance - no weights. • Easy to program.	• O(M N^3) sorting and the need for sigma_share make it slower and tuning-dependent. • Lost solutions (no elitism): P7 front has only 26 points, HV 0.636 vs 0.696 for Constrained NSGA-II (Fig. A-12). • Sharing in objective space needs normalised objectives and sigma. • Superseded by NSGA-II (which fixed exactly these three flaws).	Teaching the origin of NSGA-II.	Anything where NSGA-II is available.
13	**NPGA**	Binary tournament where each of two candidates is compared against a random comparison set (10% of N); niche count breaks ties.	• No global ranking needed, selection cost is O(t_dom) per tournament, scales to big populations. • Constrained P5 handled well: HV 1.190, 53 points (Fig. A-13). • Tournament pressure tuned by t_dom. • No fitness assignment step.	• Failed on ZDT2: 11 points, HV 0.035, IGD 0.477 (Fig. A-13). • Result depends strongly on t_dom and sigma_share (two parameters). • Not elitist; ties resolved by sharing which needs distance in objective space. • Slow in Python: 0.9 s versus 0.3 s for NSGA-II.	Very large populations where full sorting is expensive.	Small populations; tough non-convex benchmarks.

Family: Elitist EA

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
14	**NSGA-II**	Fast non-dominated sorting + crowding distance; parent+offspring union, best N survive (elitism).	• Elitism: HV of the non-dominated set never decreased (unit test 14); ZDT2 HV 0.511±0.0084 over 10 seeds (Fig. D1-P3). • Evenly spread bi-objective fronts: Spacing 0.0065 on P3 and 0.0058 on P6 (Fig. A-14). • Few parameters (pc, pm, eta); runs in 0.3 s for 10,000 evaluations. • Works with real, binary and permutation encodings (used on P8, P9).	• Constraints: without feasibility rules P5 gives Spacing 0.051 and 54 points against Constrained NSGA-II 0.0076 / 97 (Fig. A-14). • Many objectives: on DTLZ2 M=5 IGD 0.687±0.034 vs NSGA-III 0.439±0.031 (Fig. D2). • Crowding distance is a weak diversity measure in 3+ objectives. • O(M N^2) per generation.	Bi-objective problems, mixed encodings, strong default.	More than 3 objectives; tight constraints unless Deb's rules are switched on.
16	**SPEA2**	Strength S, raw fitness R = sum of strengths of dominators, density 1/(sigma_k + 2), fixed-size archive truncated by nearest-neighbour distance.	• Very even fronts: Spacing 0.0024 on P4 and 0.0028 on P5 (among the best in Task A, Fig. A-16), IGD 0.0044 on P4. • Fine-grained fitness distinguishes dominated individuals too. • Archive truncation preserves boundary points. • Archive of fixed size, elitist.	• Slow: 3.1 s per run vs 0.3 s for NSGA-II (O((N+Nbar)^2) distance work, Fig. A-16). • k-th neighbour density gets weak in many objectives. • Truncation is O(n^3) in worst case. • Archive size and population size are two parameters to match.	When evenness of the front matters (portfolios, P4).	Tight wall-clock budgets or many objectives.
17	**PESA-II**	Archive divided in hyper-grid boxes; select by box (less crowded wins), small internal population (20), archive 100.	• Best front quality among classic EAs on ZDT2: IGD 0.0092, HV 0.528 (Fig. A-17). • Region-based selection is cheap (cost in boxes, not individuals). • Small population = many generations for the same budget. • Grid gives a cheap density estimate.	• Grid resolution (divs = 10) must be set; too fine/too coarse changes diversity. • On P5 Spacing 0.0094 is less even than SPEA2 0.0028 (Fig. A-17). • Archive bounds adapt each generation, so box indices shift. • Number of boxes grows exponentially with M.	Bi-objective problems with expensive evaluations (small internal population).	Many objectives.

Family: Elitist EA (many-objective)

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
15	**NSGA-III**	NSGA-II with crowding replaced by Das-Dennis reference points: normalisation by ideal/intercepts, perpendicular association, niche-count filling.	• On 5 objectives it keeps all 69 population members spread and far outperforms NSGA-II: HV 0.490±0.095 vs 0.070±0.027, IGD 0.439±0.031 vs 0.687±0.034 (Fig. D2). • Uniform, user-controlled spread: one solution per reference line (88 points for M=3). • Hand trace in Task C shows exactly which points survive and why. • Reference points can be redefined for preference regions.	• Number of divisions sets the population: H = 4/8/12 gave IGD 0.159 / 0.099 / 0.108 (Fig. C-2). • Needs intercept normalisation, fails on degenerate or disconnected fronts (empty niches wasted). • Huge reference sets for many objectives (C(M+H-1,H)). • Slower than NSGA-II per generation (association and niching).	3-15 objectives with a regular front.	Irregular/degenerate fronts; two objectives.

Family: Decomposition

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
18	**MOEA/D**	N scalar Tchebycheff sub-problems with weight vectors, each solved cooperatively with its T nearest neighbours.	• Cheap per generation: no sorting, 1.3 s; HV 0.523, IGD 0.0092 on ZDT2. • Scales better with objectives than NSGA-II: DTLZ2 M=3 IGD 0.086 (Fig. A-18); M=5 HV 0.833±0.048 vs NSGA-II 0.070±0.027. • Good spacing on regular fronts by construction of weights. • Easy to inject preferences with weights.	• Needs a weight set and normalisation; on M=5 only 34 of 70 sub-problems end with distinct points (Fig. D2). • Poor on irregular fronts because weights do not match front geometry. • Parameters T, delta, nr affect results. • Neighbourhood updates are sequential, hard to parallelise.	Smooth regular fronts with 2-5 objectives.	Disconnected/degenerate fronts.
19	**MOEA/D-DE**	MOEA/D whose offspring is made by DE/rand/1 (F = 0.5, CR = 1) plus polynomial mutation.	• Best of all methods on ZDT2: HV 0.531±0.0037, IGD 0.0060±0.0014 over 10 seeds (Fig. D1-P3, A-19). • Handles variable linkages in ZDT2-like problems. • DE operator needs no distribution index tuning. • Inherits MOEA/D efficiency.	• On P5 IGD is 0.121±0.0082 and Spacing 0.081±0.0031 (Fig. A-19, D1-P5): uniform weights do not match the strongly skewed beam front, 25x worse than MODE. • Real-coded only (DE). • Parameters F, CR, nr, delta, T. • Penalty-based constraint handling gives uneven spread.	Continuous problems with complicated variable interactions.	Discrete or skewed-front problems.
20	**MOEA/D-ACO**	Sub-problem i owns a pheromone matrix; ant i builds a tour with tau and a weighted heuristic (1/d)^l1 (1/t)^l2; neighbours replaced as in MOEA/D.	• Natural discrete construction: always valid tours (unit test 20), HV 0.741±0.0007 on P8 (Fig. D3). • Heuristic information per weight vector biases ants to different regions of the front. • Small population (20) so budget goes to many iterations. • Cooperation via neighbours shares pheromone.	• Found only 8 distinct tours, Spacing 0.132 (Fig. A-20). • Slow: 3.0 s (Python loops) for 10,000 evaluations. • Needs problem-specific heuristics; not for continuous variables. • Parameters alpha, beta, rho, T, nr.	Routing/sequencing with good heuristic information.	Continuous problems.

Family: Constrained

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
21	**Constrained NSGA-II**	NSGA-II with Deb's rules in the crowded comparison: feasible beats infeasible, smaller violation wins, otherwise Pareto.	• Feasible, well spread results: P5 Spacing 0.0076 vs 0.051 without rules; HV 1.195 (Fig. A-21). • Parameter-free constraint handling (no penalty factor). • Final population is feasible: P7 HV 0.696, P6 HV 1.020. • Same cost as NSGA-II.	• On P7 the front is on the boundary x1+x2=90: points sit up to 0.95 units inside (Fig. A-21). • Infeasible individuals are ranked only by violation so exploration of infeasible regions near the boundary is limited. • Violation values of several constraints need scaling to be comparable. • Inherits crowding-distance weakness for M > 3.	Constrained engineering design (P5, P7).	Problems whose feasible region is tiny and disconnected (needs feasibility-first search).

Family: Swarm

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
22	**MOPSO**	Velocity update from personal best and an archive leader picked from sparse grid cells, archive of 100, turbulence mutation.	• Fast: 0.3 s; HV 1.195 on P5 and 0.966 on P4 (Fig. A-22). • Archive plus grid gives uniform spread: Spacing 0.0063. • Few operators, easy to code. • Good on smooth continuous problems.	• Constraint handling by penalty only; real-coded. • Premature convergence risk: parameters w, c1, c2, mutation decay (4) need tuning. • Leaders from sparse cells may chase unreachable regions. • Grid divisions and archive size add two more parameters.	Smooth continuous problems with cheap evaluations.	Discrete encodings without a decoder.
23	**MOGWO**	Alpha/beta/delta leaders from the archive, X = mean of three encircling moves, a decreasing 2 -> 0, archive pruned from crowded cells.	• P5: HV 1.195, IGD 0.0059 (Fig. A-23); no velocity or personal best to store. • Very few parameters (archive size, grid divisions). • Fast: 0.2 s. • Binary version through thresholding found 6 feature subsets.	• On ZDT2 it converges poorly: HV 0.320±0.057 and IGD 0.151±0.049 over 10 seeds (Fig. D1-P3). • Clipping to bounds collapsed the archive to one point - positions had to be re-sampled. • Averaging three leaders contracts diversity as a goes to 0. • Binary use needs a threshold decoder; not elitist for the pack itself.	Smooth low-dimensional problems and quick prototyping.	Concave fronts with many variables.
24	**MOABC**	Employed bees (one-dimension moves, greedy by dominance), onlookers chosen by rank, scouts reset stale sources; archive.	• Strong ZDT2 result: HV 0.530±0.0043, IGD 0.0096±0.0025 (10 seeds, Fig. D1-P3). • Scouts give built-in diversification. • Only 'limit' and colony size as parameters. • Very good Spacing on P6: 0.0030 (Fig. A-24).	• Two evaluations per source per cycle: 5 s-scale costs when objectives are expensive. • Single-dimension moves are slow on high-dimensional problems. • Dominance-based replacement is arbitrary for non-dominated pairs (50% rule here). • Real-coded only.	Continuous problems that need robust exploration (P3, P6).	High dimensional and expensive objectives.
25	**MOACO**	Two pheromone matrices (distance, time), ants with different lambda, probabilistic construction, deposits from non-dominated ants.	• Valid tours by construction; HV 0.744±0.015 on P8 and Spacing 0.101 (Fig. A-25). • Heuristic knowledge (1/d, 1/t) speeds up search. • Lambda per ant spreads solutions over the front. • Pheromone memory shares information.	• Only 10 distinct tours - low diversity on a 15-city case (Fig. A-25). • Parameters alpha, beta, rho, deposit scheme. • Slow in Python: 2.5 s. • Discrete only.	TSP/VRP type problems.	Continuous design problems.
29	**MOBA**	Frequency-driven velocity towards an archive leader, local walk with prob. 1 - r, acceptance with prob A, A decays, r grows.	• P3: HV 0.514, Spacing 0.0055 with fast runs 0.2 s (Fig. A-29). • Automatic switch exploration -> exploitation through A and r. • P4: HV 0.961. • Few lines of code.	• Needs 5 parameters (fmin, fmax, alpha, gamma, r0); alpha = 0.9 froze the swarm in my first trial, 0.97 was needed. • IGD 0.014 on P4 is about 3x worse than SPEA2 0.0044 (Fig. A-29). • Loudness-based acceptance throws away good moves late. • Real-coded only.	Quick continuous experiments.	Tight budgets; problems needing fine tuning.

Family: Physics-inspired

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
26	**MOSA**	Gaussian neighbour moves with temperature-dependent size, Boltzmann acceptance on a random weighted change, geometric cooling, archive.	• Extremely light: 0.6 s; P5 IGD 0.0051, Spacing 0.0037 (Fig. A-26). • Can escape local optima (accepts worse moves). • No population-sorting, tiny memory. • Works with any move operator.	• Weak on ZDT2 with many variables: HV 0.441±0.028 over 10 seeds vs 0.531±0.0037 for MOEA/D-DE (Fig. D1-P3). • Cooling schedule T0, Tend, move size - sensitive. • Slow convergence; budget is split among chains. • Random weights give uneven coverage.	Rugged landscapes with cheap neighbours and small memory.	Smooth high-dimensional continuous problems.

Family: Evolutionary (DE)

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
27	**MODE**	DE/rand/1/bin; the trial vector replaces its target only if it dominates it; non-dominated archive.	• P5: HV 1.195±0.0000, IGD 0.0048±0.0001 over 10 seeds - among the best (Fig. D1-P5). • Only F, CR, N to set; no distribution indices. • Self-scaling step from population differences. • Trace in Task C reproduces every number.	• Collapsed to 1 point on ZDT2, HV 0.110: strict dominance replacement drives the population to the (0,1) end (Fig. A-27, C-3). • Nothing maintains diversity except the archive, which cannot store unvisited points. • Replacement needs strict dominance, so mutually non-dominated trials are wasted. • Real-coded only.	Constrained smooth problems where the front is convex-ish (P5).	Concave fronts.

Family: Human-inspired

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
28	**MO-TLBO**	Teacher = non-dominated member; teaching phase X + r(T - TF*mean) with TF shrinking 2 -> 1; learner phase towards the better partner; archive.	• No algorithm-specific parameters except population size (TF is scheduled). • P5: HV 1.195, Spacing 0.021 (Fig. A-28). • Two phases give both exploration and refinement. • Simple to code.	• Slow on ZDT2: HV 0.337±0.132, IGD 0.187±0.162 over 10 seeds (Fig. D1-P3). • Two evaluations per learner per iteration (halves the iterations). • Teacher chosen from a small non-dominated set reduces diversity. • Real-coded only.	Parameter-free first trials on smooth problems.	Concave, many-variable problems on small budgets.

Family: Hybrid (memetic)

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
30	**MOMA**	NSGA-II plus hill-climbing local search (random-weight scalarisation) on 20% of the offspring.	• P5: HV 1.195, IGD 0.0068 (Fig. A-30). • Works on binary P9 feature selection (5 subsets, Fig. A-30) and permutations (P8). • Local search refines solutions per evaluation. • Inherits NSGA-II elitism.	• High variance on ZDT2: HV 0.378±0.112 (10 seeds) vs NSGA-II 0.511±0.0084 because evaluations go to the local search. • Extra parameters p_ls, steps; budget share of local search. • Random-weight scalar search can pull solutions to the ends. • Each iteration costs ~1.6x NSGA-II evaluations.	Rugged discrete problems (P8, P9) where neighbourhood moves help.	Smooth problems where plain NSGA-II converges anyway.

Family: Many-objective

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
31	**RVEA**	Reference vectors, angle-penalised distance APD, one survivor per vector, vector adaptation.	• M=5 DTLZ2: HV 0.901±0.045 and IGD 0.336±0.020, second only to HypE but about 500 times cheaper (0.1 s vs 59.2 s, Fig. D2). • Very fast: 0.1 s for a 10,000-evaluation run on M=5. • Penalty angle grows over time: exploration first, convergence later. • Adaptation follows the front's scale.	• Population shrinks to 56 points on M=3 because empty vectors discard individuals. • Needs a known number of generations (t_max) for the penalty schedule. • Parameters alpha, fr. • Poor on irregular fronts.	4-10 objectives with regular fronts.	Irregular fronts or 2 objectives.

Family: Indicator-based

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
32	**IBEA**	Fitness = sum of -exp(-I(y,x)/(c k)) with the additive epsilon indicator; iteratively remove the worst; no diversity step.	• Excellent on P14: HV 0.704, Spacing 0.0036 (Fig. A-32). • No explicit diversity mechanism needed: the indicator already prefers spread. • Works for any number of objectives. • ZDT2 HV 0.521±0.0081 (10 seeds).	• Constraint handling only by penalty; P5 IGD 0.202±0.020 (10 seeds) is the worst of the EAs (Fig. D1-P5). • Kappa must be tuned; fitness is O(N^2). • Needs normalised objectives. • Fitness removal loop is sequential.	Unconstrained problems wanting a well converged, even set.	Skewed constrained problems.
33	**SMS-EMOA**	Steady-state (mu+1); discard the point with the smallest hypervolume contribution in the worst front.	• Best Spacing in Task A on P14: 0.0008 and top IGD 0.0035 (Fig. A-33). • Directly optimises hypervolume, so HV is close to the best on ZDT2: 0.512±0.0095 vs 0.531±0.0037 for the best method (10 seeds). • Steady-state: always keeps all best solutions. • No diversity parameter.	• Slowest EA here: 8.0 s for 10,000 evaluations (one sort per evaluation). • Exact HV contribution is exponential in M; MC needed beyond 3 objectives. • Reference point choice affects results. • Hard to parallelise.	Bi/tri-objective problems with expensive evaluations.	Many objectives or when wall-clock matters.
34	**HypE**	Monte-Carlo estimate of hypervolume contributions with the HypE weights alpha_i; remove worst repeatedly.	• Best on 5 objectives: HV 1.119±0.020 and IGD 0.306±0.0063 vs NSGA-III 0.490±0.095 (Fig. D2); MC estimation keeps hypervolume selection tractable. • M=3: HV 0.696, the highest of the four P10 runs in Task A. • Sample count trades accuracy for time. • No diversity parameter.	• Very slow: 24.8 s on M=3 and 59.2 s on M=5 (Fig. A-34, D2). • Estimates are noisy: unit test needs 200000 samples for 5% accuracy. • Number of samples S is a parameter. • IGD on M=3 is 0.129 (MOEA/D 0.086).	Many objectives when HV-driven selection is wanted.	Strict time limits.

Family: Preference-based

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
35	**R-NSGA-II**	NSGA-II whose last front is ranked by distance to decision-maker reference points with epsilon-clearing.	• Focuses the population: on P14 69% of the points lie within 0.15 kg of 1.8 kg vs 46% for NSGA-II (Fig. E-5). • Saves evaluations on the region of interest. • Several reference points are allowed. • Same cost as NSGA-II.	• A wrong reference point steers the search away from what is wanted; on P14 the reference lies behind the front. • Epsilon needs tuning (0.01-0.02). • Less coverage of the rest of the front. • P5: needs an invented reference (1.194 HV).	Decision maker can name aspirations (P14).	No preference information available.

Family: Gradient-based

#	Algorithm	What it does (my words)	Advantages	Disadvantages	Best choice when	Avoid when
36	**MGDA**	Min-norm point of the convex hull of task gradients gives a common descent direction (Frank-Wolfe solver).	• Guaranteed descent for every task: <d*,g_i> >= ||d*||^2 (unit test 36). • Reaches the Pareto set: P13a ends at (2.5, 2.5) on the segment x1 = x2 (Fig. A-36). • No loss weights to choose. • Stops at Pareto-stationary points (d* = 0).	• Needs gradients of all tasks every step (M backward passes). • Gives one Pareto point per run - the point depends on start (2.5,2.5 from (0,5)). • Tends to follow tasks with small gradients; sensitive to gradient scales (needs normalisation in the network). • Minimum-norm QP per step.	Differentiable multi-task learning (P13).	Black-box or non-differentiable problems.
37	**GradNorm**	Adaptive task weights w_i so that w_i||grad L_i|| follows the mean gradient norm times relative inverse training rate^alpha.	• Balances tasks automatically: toy weights end near (1.63, 0.37) summing to 2 (Fig. A-37). • On the network with regression targets x10 both losses still fall (Fig. A-P13b). • Single backward pass per task for the shared layer norm. • Light overhead.	• Hyper-parameters alpha, lr_w; weights hit the floor 0.01 when one task's gradient is huge (regression x10: w = (1.99, 0.01)). • Final regression loss ratio 0.112 (GradNorm) vs 0.015 (equal) is worse than equal weights in that case. • Heuristic: no Pareto guarantee. • Needs a chosen shared layer.	Multi-task networks with unbalanced loss scales.	When Pareto optimality guarantees are required.

How the families differ (diversity, convergence, constraints, non-convex fronts)

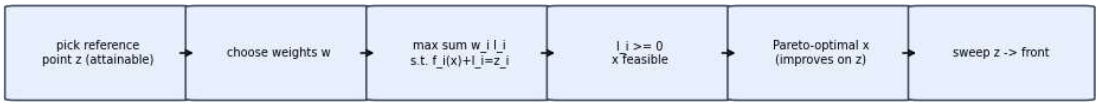
Diversity: classical methods get diversity only from the sweep (weights, eps, z) and therefore inherit its unevenness; non-elitist EAs rely on sharing or sub-populations and lose it quickly (VEGA collapsed to one point on ZDT2); elitist EAs add an explicit density estimate (crowding distance, k-th neighbour, grid boxes) and give the most even bi-objective fronts, SPEA2 and SMS-EMOA being the most even; decomposition and many-objective methods (MOEA/D, NSGA-III, RVEA) get diversity from the geometry of weights or reference vectors, which is superb on regular simplex-like fronts and wasted on skewed ones; indicator-based methods let the indicator itself prefer spread. Convergence: gradient and scalarisation methods converge fastest per solve on smooth convex problems; DE-based and decomposition EAs (MOEA/D-DE, MODE on P5) converge best among EAs on continuous problems, while the weaker swarm variants (MOGWO, MO-TLBO, MOSA) needed far more evaluations on ZDT2. Constraints: classical methods pass constraints to a solver, Deb's rules (Constrained NSGA-II) are the cleanest EA mechanism and the others use a penalty; ignoring them returned a front where 45% of the points violate the stress limit of P5. Non-convex fronts: weighted sum and p = 1 fail by construction; eps-constraint, Tchebycheff, Benson with a tight reference point, and every Pareto-based EA reach the dent. More than three objectives: dominance-based selection and crowding degrade (on random DTLZ2 populations the non-dominated share grows from 35% at M=3 to 57% at M=5 and 70% at M=8), so reference-point (NSGA-III, RVEA, MOEA/D) and indicator (HypE) methods are needed.

5. Task C — deep dive on my three algorithms (R = 40: algorithms 3, 15, 27)

5.1 Algorithm 3 — Benson's Method

Study card

Full form, year, authors	Benson's method (H. P. Benson, 1978, 'Existence of efficient solutions for vector maximization problems', J. Optim. Theory Appl. 26). Often called Benson's test/proper-efficiency method.
Problem it fixes	Weighted sum cannot reach non-convex parts and cannot tell whether a given design is already Pareto-optimal. Benson starts from an attainable reference point z and finds a Pareto-optimal point that improves on it, with one LP/NLP.
Core idea (≤5 lines)	Pick a reference point z that some feasible design dominates or equals. Introduce slacks l_i = z_i - f_i(x) ≥ 0 (how much each objective is improved) and maximise their weighted sum. The optimal x* is Pareto-optimal and dominates z; if every l_i = 0, z itself was Pareto-optimal. Sweeping z (and w) draws the front.
Numbered steps	1. Choose z (objective vector of a feasible design or an aspiration) and weights w_i > 0. 2. Solve max Σ w_i l_i s.t. f_i(x) + l_i = z_i, l_i ≥ 0, x in X (SLSQP, 6 random starts). 3. Report x*, f(x*), l*. 4. If l* = 0 the point z is Pareto-optimal; otherwise f(x*) improves z by l*. 5. Repeat with a new z (the sweep z_k = (t_k, 1-t_k) + delta in normalised space).
Parameters	**z**: beyond the nadir the constraints are inactive and Benson collapses to weighted sum (offset 0.6: 2 end points); too tight (no feasible design improves it) the sub-problem is infeasible (offset 0.02 on ZDT2: 2 points). Offset 0.2 gave the best coverage (Fig. C-1). **w**: with equal weights on a linear front (P7) the optimum is a degenerate tie; unequal weights pick a vertex.
Convergence / diversity	Each solve is exact (local optimum of a smooth NLP; LP optimum for linear problems). Diversity comes only from the user's sweep of z and w; nothing spreads the points automatically.
Constraints / non-convex / M>3	Constraints are passed to the solver. Non-convex fronts are reachable when z is attainable and close (Fig. C-1). For M objectives M slacks are added; the sweep needs a grid of dimension M-1, so it is practical for 2-3 objectives.
Complexity	One NLP (SLSQP: about O((n+M)^3) per iteration) or LP per reference point; k reference points cost k solves, no population.



Hand trace on P1 (min f1 = (x-3)^2, f2 = (x-7)^2, w = (1,1)). Maximising l1 + l2 = z1 + z2 - [(x-3)^2 + (x-7)^2] means minimising S(x) = (x-3)^2 + (x-7)^2, whose unconstrained minimiser is x = 5 (S' = 4x - 20 = 0). The constraints l ≥ 0 restrict x to (x-3)^2 ≤ z1 and (x-7)^2 ≤ z2, i.e. x in [max(3-√z1, 7-√z2), min(3+√z1, 7+√z2)]. The solution is x = 5 clipped to that interval.

	Setting 1: z = (10, 8)	Setting 2: z = (2, 20)
allowed interval for x	[4.1716, 6.1623]	[2.5279, 4.4142]
x* (hand)	5 inside the interval -> x = 5.0000	5 is above 4.4142 -> clipped x = 3+√2 = 4.4142
f(x*) hand	(4.0000, 4.0000)	(2.0000, 6.6863)
l = z - f hand	(6.0000, 4.0000), sum 10.0000	(0.0000, 13.3137), sum 13.3137
x*, f, l from my code (benson())	x = 5.0000, f = (4.0000, 4.0000), l = (6.0000, 4.0000)	x = 4.4142, f = (2.0000, 6.6863), l = (0.0000, 13.3137)
code reproduces hand trace?	**YES**	**YES**

Second setting in words: with z1 = 2 the first objective is capped, (x-3)^2 ≤ 2 gives x ≤ 3 + 1.4142 = 4.4142, which is below the sum-minimiser 5, so the constraint f1 ≤ z1 is active (l1 = 0) and all improvement goes to f2: l2 = 20 - 6.6863 = 13.3137.

Sensitivity study: reference-point offset delta (the most important parameter)

delta in z_k = (t_k, 1-t_k) + delta	distinct non-dominated points	points with 0.2 < f1 < 0.8	HV (mean ± std)	IGD (mean ± std)
0.02	2.0	0.0	0.210 ± 0.0000	0.354 ± 0.0000
0.2	11.0	5.0	0.488 ± 0.0003	0.048 ± 0.0012
0.6	2.2	0.2	0.217 ± 0.0136	0.334 ± 0.0400

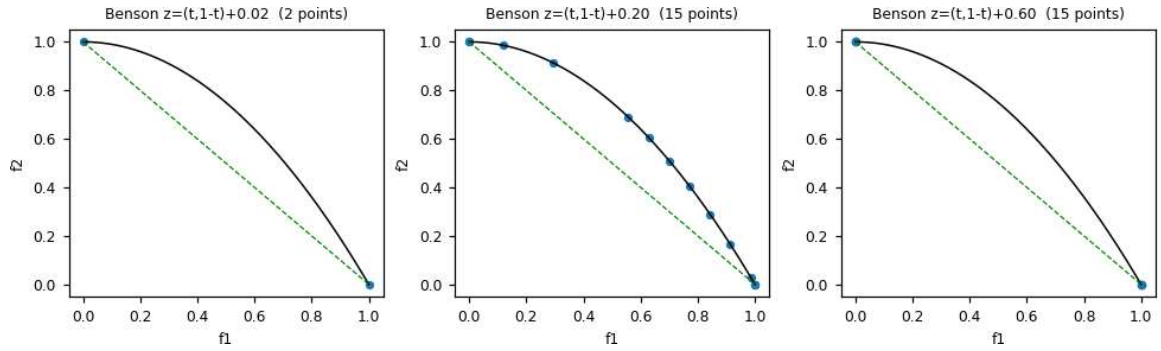


Fig. C-1: Benson on ZDT2 for three offsets (15 reference points each; green dashed = the line f1 + f2 = 1; 10 seeds only change the multi-start, so the std is tiny). Because the ZDT2 front lies *above* that line, a reference point on it with offset 0.02 is not attainable in the middle (infeasible sub-problems, only the two end points remain); offset 0.2 is attainable and tight enough to bind, so Benson finds the dented middle (5 points, best HV 0.488, IGD 0.048); offset 0.6 is beyond the nadir, the constraints are inactive and the method reproduces weighted sum (end points only).

5.2 Algorithm 15 — NSGA-III

Study card

Full form, year, authors	Non-dominated Sorting Genetic Algorithm III — K. Deb and H. Jain, 2014, IEEE Trans. Evolutionary Computation 18(4) (reference-point based many-objective NSGA).
Problem it fixes	In NSGA-II the crowding distance stops working with 4+ objectives (almost every point is non-dominated and neighbours are no longer meaningful); NSGA-III replaces it by niching around well-spread reference points.
Core idea (≤5 lines)	Keep NSGA-II's elitist (P+Q) non-dominated sorting. Choose structured reference points on the unit simplex (Das-Dennis). When the last front must be cut, normalise the objectives, associate every candidate with its nearest reference line and fill the missing slots from the least-crowded reference lines, preferring the candidate closest to the line.
Numbered steps	1. Reference points Z = Das-Dennis(M,H), |Z| = C(M+H-1,H); N ≈ |Z|. 2. R = P u Q; non-dominated sort; take whole fronts until size ≥ N; last front F_l. 3. Ideal z_i = min f_i over S_t; translate f'_i = f_i - z_i. 4. Extreme points by ASF(f',w) = max_i f'_i / w_i with w = e_i (+1e-6); intercepts a_i from the hyperplane through them; f~_i = f'_i / a_i. 5. Association: d⊥(s,w) = ||f~ - (w·f~/||w||^2) w||, j(s) = argmin_w d⊥. 6. Niche count rho_j = number of already chosen members at line j. 7. Repeat K = N - |chosen| times: pick the line with minimum rho (random tie); if rho = 0 take the F_l member with smallest d⊥ else a random member; rho_j += 1.
Parameters	**H** (divisions): too small gives very few lines and small N (H = 4: 15 lines, N = 16, IGD 0.159), too large gives a big N and few generations for the same budget (H = 12: N = 92, 10 generations fewer, HV 0.590 ± 0.035, noisy); H = 8 was best (IGD 0.099) (Fig. C-2). pc, eta_c, eta_m: standard SBX/polynomial mutation (0.9, 15, 20).
Convergence / diversity	Elitist non-dominated sorting for convergence; diversity is enforced by the niche counts, so every reference line gets a representative if one exists. At 10,000 evaluations it is still converging on DTLZ2 (IGD 0.120 at 10k, 0.055 at 30k evaluations, best of NSGA-II/MOEA/D at 30k).
Constraints / non-convex / M>3	Constraints: Deb's constrained domination can be put into the sort (here the same penalty as the other methods). Non-convex fronts: dominance-based, so fine. M>3: this is what it was designed for (M=5 in Task D). Weak point: irregular or degenerate fronts leave reference lines empty.
Complexity	O(N^2 M) for the sort of 2N points plus O(N |Z| M) for association per generation, i.e. a little more than NSGA-II.



Hand trace: tiny instance on P1 (one variable x, minimise (x-3)^2 and (x-7)^2), M = 2, H = 3, reference points Z = {(0,1), (1/3,2/3), (2/3,1/3), (1,0)}, N = 4. Parent population x = {1.0, 3.5, 6.0, 9.0}. To make every number checkable the offspring are given (they stand for the result of SBX + mutation, which is verified separately by unit tests): generation 1 offspring {1.5, 8.0, 2.0, 5.5}, generation 2 offspring {9.0, 4.5, 4.0, 5.0}. In the tables 'idx' numbers the 8 combined individuals (0-3 parents, 4-7 offspring). The hand trace was computed with plain Python loops (nsga3_trace in run_task_c.py) and compared with my module's nsga3_select for 30 different random seeds.

Generation 1

idx	x	f1	f2	front	f'~1 (normalised)	f'~2	nearest line j (Z_j)	d_perp	selected
0	1.0	4.00	36.00	4	-	-	-	-	no
1	3.5	0.25	12.25	1	0.000	1.000	0 (0.00,1.00)	0.000	**yes**
2	6.0	9.00	1.00	1	1.000	0.000	3 (1.00,0.00)	0.000	**yes**
3	9.0	36.00	4.00	3	-	-	-	-	no
4	1.5	2.25	30.25	3	-	-	-	-	no
5	8.0	25.00	1.00	2	2.829	0.000	3 (1.00,0.00)	0.000	no
6	2.0	1.00	25.00	2	0.086	2.133	0 (0.00,1.00)	0.086	**yes**
7	5.5	6.25	2.25	1	0.686	0.111	3 (1.00,0.00)	0.111	**yes**

Fronts: [[1, 2, 7], [5, 6], [3, 4], [0]]. Whole fronts taken before the cut: [1, 2, 7]; last front F_l = [5, 6]. Ideal point (0.25, 1.00); extreme points = idx [2, 1]; intercepts (8.750, 11.250). Initial niche counts rho = [1, 0, 0, 2]. Niching step: line 0 (rho = 1, 1 tied lines) → idx 6 (rho>0 -> random member). **Selected idx = [1, 2, 6, 7]** (x = [3.5, 6.0, 2.0, 5.5]) and my code returns [1, 2, 6, 7]: **MATCH**.

Generation 2

idx	x	f1	f2	front	f'~1 (normalised)	f'~2	nearest line j (Z_j)	d_perp	selected
0	3.5	0.25	12.25	1	0.000	1.000	0 (0.00,1.00)	0.000	**yes**
1	6.0	9.00	1.00	1	1.000	0.000	3 (1.00,0.00)	0.000	**yes**
2	5.5	6.25	2.25	1	0.686	0.111	3 (1.00,0.00)	0.111	no
3	2.0	1.00	25.00	2	-	-	-	-	no
4	9.0	36.00	4.00	2	-	-	-	-	no
5	4.5	2.25	6.25	1	0.229	0.467	1 (0.33,0.67)	0.004	**yes**
6	4.0	1.00	9.00	1	0.086	0.711	0 (0.00,1.00)	0.086	no
7	5.0	4.00	4.00	1	0.429	0.267	2 (0.67,0.33)	0.047	**yes**

Fronts: [[0, 1, 2, 5, 6, 7], [3, 4]]. Whole fronts taken before the cut: []; last front F_l = [0, 1, 2, 5, 6, 7]. Ideal point (0.25, 1.00); extreme points = idx [1, 0]; intercepts (8.750, 11.250). Initial niche counts rho = [0, 0, 0, 0]. Niching step: line 0 (rho = 0, 4 tied lines) → idx 0 (rho=0 -> closest to line). Niching step: line 1 (rho = 0, 3 tied lines) → idx 5 (rho=0 -> closest to line). Niching step: line 2 (rho = 0, 2 tied lines) → idx 7 (rho=0 -> closest to line). Niching step: line 3 (rho = 0, 1 tied lines) → idx 1 (rho=0 -> closest to line). **Selected idx = [0, 1, 5, 7]** (x = [3.5, 6.0, 4.5, 5.0]) and my code returns [0, 1, 5, 7]: **MATCH**.

Sensitivity study: number of divisions H on DTLZ2 (M = 3), 10 seeds each, 10,000 evaluations

H	reference points	N	HV mean ± std	IGD mean ± std	Spacing	points
4	15	16	0.630 ± 0.004	0.159 ± 0.006	0.131 ± 0.017	15.9
8	45	48	0.639 ± 0.010	0.099 ± 0.004	0.081 ± 0.007	47.6
12	91	92	0.590 ± 0.035	0.108 ± 0.013	0.060 ± 0.006	90.6

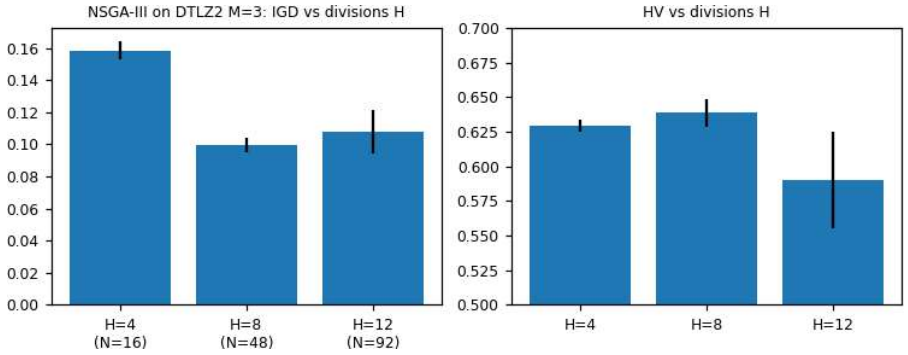


Fig. C-2: NSGA-III sensitivity to H. A very coarse set (H=4) is fast to converge but returns only 16 points (IGD 0.159); H=8 balances density and generations; H=12 gives the densest and most even set (Spacing 0.060) but, with a fixed 10,000-evaluation budget, fewer generations and a noisier HV.

5.3 Algorithm 27 — MODE (multi-objective differential evolution)

Study card

Full form, year, authors	Multi-Objective Differential Evolution. DE: R. Storn and K. Price (1997); Pareto-based MODE: F. Xue, A. C. Sanderson and R. J. Graves, 2003 (IEEE CEC). The lecture form implemented here: DE/rand/1/bin where the trial replaces its target only if it dominates it, plus a non-dominated archive.
Problem it fixes	Genetic algorithms need distribution indices and crossover/mutation tuning; DE uses scaled differences of population members so the step size adapts automatically. MODE brings that to Pareto problems by replacing the scalar 'better' test by dominance.
Core idea (≤5 lines)	For each target x_i make a mutant v = x_r1 + F (x_r2 - x_r3) from three other members, mix v with x_i by binomial crossover (rate CR, one forced coordinate j_rand) to get the trial u, evaluate u and let it replace x_i only if u dominates x_i. All non-dominated points seen are kept in an archive (crowding-truncated to 100).
Numbered steps	1. Random population of N vectors in the box; evaluate. 2. For i = 1..N: choose distinct r1, r2, r3 ≠ i. 3. v = x_r1 + F (x_r2 - x_r3). 4. u_k = v_k if rand_k < CR or k = j_rand, else x_ik; clip u to the bounds. 5. Evaluate u. 6. If u dominates x_i then x_i = u (otherwise keep x_i). 7. Add all trials to the archive; repeat 2-7 until the budget is spent.
Parameters	**F** (scale factor): too small (0.2) means tiny steps, slow movement; too large (0.9) means overshoot and noisy search. **CR**: low = coordinate-wise moves, high = rotation-invariant moves. **N**: needs ≥ 4; small N has little difference-vector variety, large N is slow. On P5 the effect of F is small (Fig. C-3); on ZDT2 the algorithm collapses for every F.
Convergence / diversity	Convergence is fast and monotone for every individual (an individual never gets worse in dominance). Diversity is weak: nothing pushes members apart and mutually non-dominated trials are wasted, so the population can drift to one end of the front.
Constraints / non-convex / M>3	Constraints: penalty (here) or Deb rules in the replacement test. Non-convex: on ZDT2 the (0,1) end point dominates its neighbours' trials in f2 only after g shrinks, and the population contracts onto it (HV 0.110, 1 point, all 10 seeds). M>3: dominance becomes rare in the replacement test so improvement stalls.
Complexity	Per generation O(N n) for variation, N evaluations, O(N M) dominance tests, plus archive maintenance O(A^2 M); memory O(N n + A n).



Hand trace: P14 (two variables x1 in [13,17], x2 in [40,90]; minimise weight f1 = 0.4 + 0.05 x1 + 0.012 x2 and -runtime f2 = -0.2 x2 / (0.5 + 0.03 x1)), N = 4, F = 0.5, CR = 0.9, seed 7. The random numbers (initial population, the indices r1, r2, r3 for each target, the crossover uniforms and j_rand) are those produced by numpy's generator with seed 7; the arithmetic below was recomputed independently with plain Python and then compared with my module's MODE.iterate (population after each iteration identical: iteration 1 **MATCH**, iteration 2 **MATCH**).

Initial population

i	x1	x2	f1 (kg)	f2 (-hours)
0	15.5004	84.8607	2.1933	-17.5875
1	16.1027	51.2604	1.8203	-10.4285
2	14.2007	83.6777	2.1142	-18.0725
3	13.0211	81.0614	2.0238	-18.2031

Iteration 1

i	r1,r2,r3	mutant v = x_r1 + 0.5 (x_r2 - x_r3)	j_rand	crossover (x1,x2 from v?)	trial u (clipped)	f(u)	f(x_i)	u dominates x_i?
0	2,3,1	(12.6598, 98.5782)	1	yes, yes	(13.0000, 90.0000)	(2.1300, -20.2247)	(2.1933, -17.5875)	**replace**
1	3,0,2	(13.6709, 81.6529)	1	yes, no	(13.6709, 51.2604)	(1.6987, -11.2644)	(1.8203, -10.4285)	**replace**
2	1,0,3	(17.3424, 53.1600)	1	yes, yes	(17.0000, 53.1600)	(1.8879, -10.5267)	(2.1142, -18.0725)	**keep**
3	0,2,1	(14.5493, 101.0693)	2	yes, yes	(14.5493, 90.0000)	(2.2075, -19.2209)	(2.0238, -18.2031)	**keep**

Population after iteration 1: x0 = (13.000, 90.000); x1 = (13.671, 51.260); x2 = (14.201, 83.678); x3 = (13.021, 81.061). Archive size 3.

Iteration 2

i	r1,r2,r3	mutant v = x_r1 + 0.5 (x_r2 - x_r3)	j_rand	crossover (x1,x2 from v?)	trial u (clipped)	f(u)	f(x_i)	u dominates x_i?
0	1,2,3	(14.2607, 52.5685)	1	yes, yes	(14.2607, 52.5685)	(1.7439, -11.3316)	(2.1300, -20.2247)	**keep**
1	0,3,2	(12.4102, 88.6919)	2	yes, yes	(13.0000, 88.6919)	(2.1143, -19.9308)	(1.6987, -11.2644)	**keep**
2	3,0,1	(12.6856, 100.4312)	2	yes, yes	(13.0000, 90.0000)	(2.1300, -20.2247)	(2.1142, -18.0725)	**keep**
3	1,2,0	(14.2713, 48.0992)	2	yes, yes	(14.2713, 48.0992)	(1.6908, -10.3647)	(2.0238, -18.2031)	**keep**

Population after iteration 2: x0 = (13.000, 90.000); x1 = (13.671, 51.260); x2 = (14.201, 83.678); x3 = (13.021, 81.061). Archive size 6.

Reading the trace: target 0 is replaced because its trial hit the corner (13, 90) (lightest-weight design at maximum battery is not required to dominate in both... it dominates since f1 2.130 < 2.193 and -20.22 < -17.59); target 2's trial is clipped at x1 = 17 and is dominated in runtime so it is rejected; this shows the key rule 'replace only if it dominates' and the clipping rule.

Sensitivity study: scale factor F (0.2, 0.5, 0.9), CR = 0.9, N = 100, 10 seeds, 10,000 evaluations

problem / F	HV mean ± std	IGD mean ± std	Spacing	points in archive
P3 F0.2	0.099 ± 0.0330	0.6198 ± 0.0298	0.0000 ± 0.0000	1.0
P3 F0.5	0.110 ± 0.0001	0.6098 ± 0.0000	0.0000 ± 0.0000	1.0
P3 F0.9	0.110 ± 0.0000	0.6098 ± 0.0000	0.0000 ± 0.0000	1.0
P5 F0.2	1.195 ± 0.0002	0.0058 ± 0.0022	0.0122 ± 0.0097	100.0
P5 F0.5	1.195 ± 0.0000	0.0048 ± 0.0001	0.0045 ± 0.0012	100.0
P5 F0.9	1.195 ± 0.0000	0.0049 ± 0.0001	0.0038 ± 0.0005	100.0

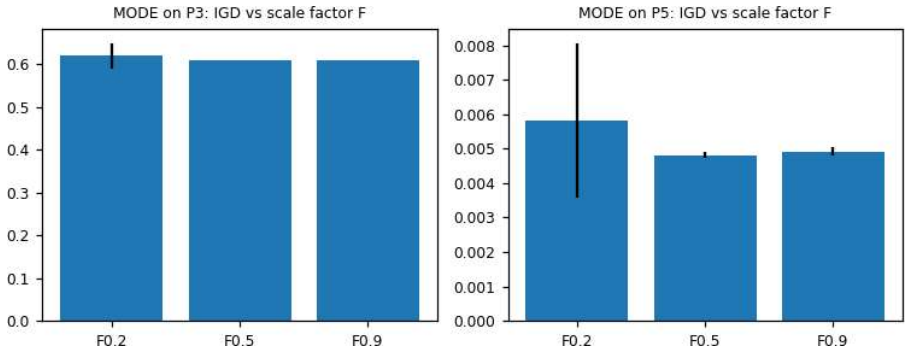


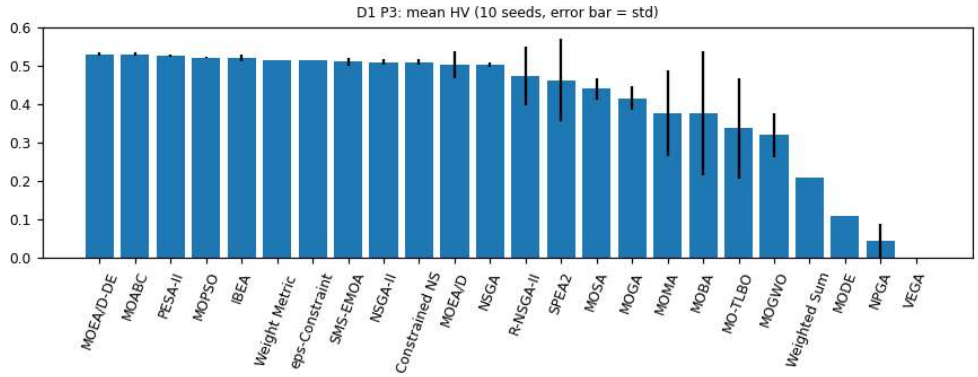
Fig. C-3: MODE sensitivity. On P5 (convex-like, constrained) F barely changes the front (IGD about 0.005-0.006) but larger F is slightly more accurate and more consistent; on ZDT2 every F collapses to the single end point (0,1): the failure comes from the strict dominance replacement rule, not from the step size.

6. Task D — head-to-head comparison (own implementations, equal budget 10,000 evaluations, 10 seeds)

D1(a) P3 ZDT2 (unconstrained, n = 10)

rank by HV	algorithm	HV (mean ± std)	IGD (mean ± std)	Spacing (mean ± std)	points	time s
1	19 MOEA/D-DE	0.531 ± 0.004	0.0060 ± 0.0014	0.0035 ± 0.0006	96	1.0
2	24 MOABC	0.530 ± 0.004	0.0096 ± 0.0025	0.0113 ± 0.0036	87	0.3
3	17 PESA-II	0.527 ± 0.003	0.0102 ± 0.0011	0.0087 ± 0.0024	100	1.2
4	22 MOPSO	0.522 ± 0.002	0.0114 ± 0.0007	0.0074 ± 0.0010	100	0.3
5	32 IBEA	0.521 ± 0.008	0.0133 ± 0.0040	0.0109 ± 0.0045	74	0.4
6	4 Weight Metric (p-norm, Tchebycheff) *	0.517 ± 0.000	0.0180 ± 0.0000	0.0108 ± 0.0000	21	0.0
7	2 eps-Constraint *	0.514 ± 0.000	0.0389 ± 0.0000	0.0605 ± 0.0000	19	0.0
8	33 SMS-EMOA	0.512 ± 0.010	0.0169 ± 0.0057	0.0080 ± 0.0023	100	7.5
9	14 NSGA-II	0.511 ± 0.008	0.0180 ± 0.0050	0.0061 ± 0.0008	99	0.5
10	21 Constrained NSGA-II	0.511 ± 0.008	0.0180 ± 0.0050	0.0061 ± 0.0008	99	0.3
11	18 MOEA/D	0.504 ± 0.034	0.0214 ± 0.0248	0.0090 ± 0.0047	74	1.3
12	12 NSGA	0.503 ± 0.005	0.0236 ± 0.0037	0.0182 ± 0.0053	46	0.1
13	35 R-NSGA-II	0.473 ± 0.077	0.0485 ± 0.0651	0.0076 ± 0.0021	96	0.4
14	16 SPEA2	0.464 ± 0.108	0.0642 ± 0.1060	0.0102 ± 0.0139	91	0.9
15	26 MOSA	0.441 ± 0.028	0.0695 ± 0.0170	0.0486 ± 0.0147	25	0.2
16	11 MOGA	0.416 ± 0.031	0.0700 ± 0.0221	0.0240 ± 0.0073	33	0.1
17	30 MOMA	0.378 ± 0.112	0.1359 ± 0.1661	0.0148 ± 0.0086	59	0.2
18	29 MOBA	0.377 ± 0.163	0.1735 ± 0.1981	0.0075 ± 0.0065	75	0.2
19	28 MO-TLBO	0.337 ± 0.132	0.1875 ± 0.1623	0.0172 ± 0.0122	100	0.5
20	23 MOGWO	0.320 ± 0.057	0.1511 ± 0.0491	0.0520 ± 0.0273	98	0.2
21	1 Weighted Sum *	0.210 ± 0.000	0.3538 ± 0.0000	0.0000 ± 0.0000	2	0.0
22	27 MODE	0.110 ± 0.000	0.6098 ± 0.0000	0.0000 ± 0.0000	1	0.2
23	13 NPGA	0.044 ± 0.046	0.5202 ± 0.1066	0.0700 ± 0.0705	9	0.7
24	10 VEGA	0.000 ± 0.000	0.9198 ± 0.1146	0.0017 ± 0.0052	2	0.0

* classical sweeps (21 weights / eps / Tchebycheff weights) are deterministic up to multi-start, so std = 0 and no time measured per run. True-front HV for reference: 0.542.



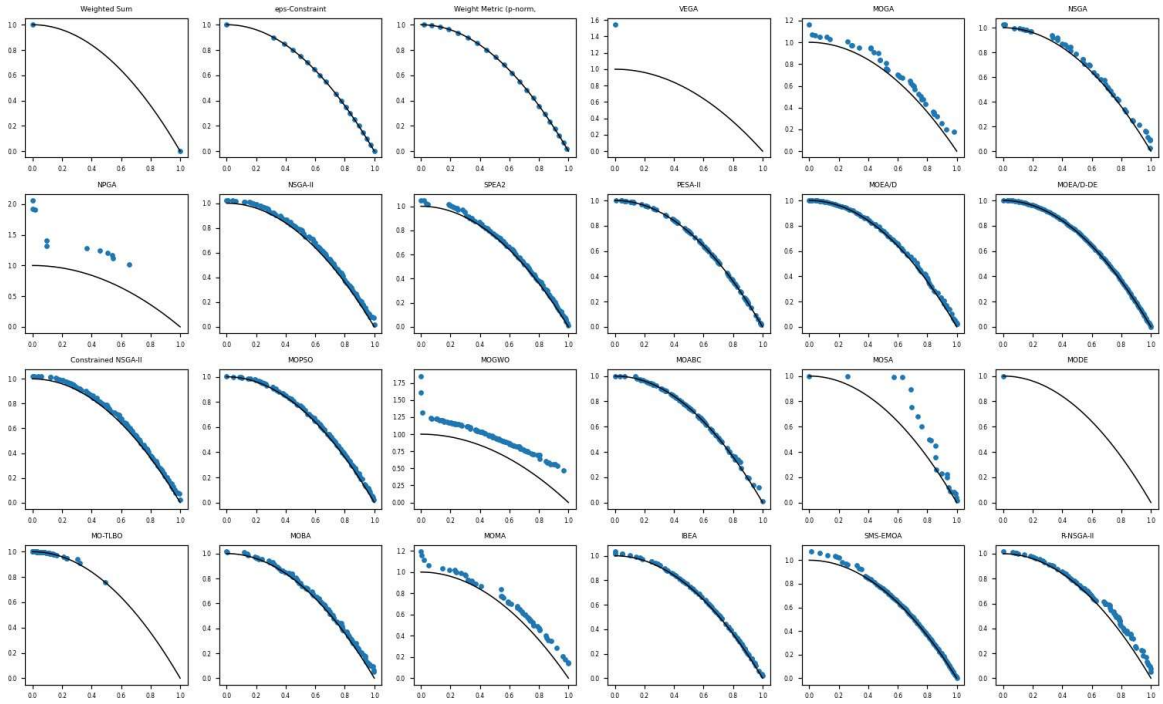


Fig. D-1a: fronts of all algorithms on P3, seed 1 (black line = true front).

Why this ranking (P3). The first five (MOEA/D-DE, MOABC, PESA-II, MOPSO, IBEA) all reach HV 0.52–0.53 of a possible 0.542 and IGD 0.006–0.013, and their HV differences are within about 1–2 standard deviations, so they should be read as a tie group with MOEA/D-DE slightly ahead in both HV and IGD (0.006): its DE operator moves the nine g-variables together towards 0 quickly while the weight vectors keep the points spread. NSGA-II, Constrained NSGA-II (identical here: no constraints), SMS-EMOA and the Tchebycheff sweep follow (HV 0.511–0.517): NSGA-II is evenly spread (Spacing 0.006) but its 100 points are still slightly above the front (IGD 0.018) after only 99 generations. The 21-point Tchebycheff sweep ranks above NSGA-II in HV although it has only 21 points, because each point is an exact optimum; its IGD (0.018) equals NSGA-II's. The eps-constraint sweep is exact too but unevenly spaced (Spacing 0.061, IGD 0.039). SPEA2 (0.464 ± 0.108) and R-NSGA-II (0.473 ± 0.077) have large standard deviations (some seeds converged less well within the budget). MOSA, MOGA, MOMA, MOBA, MO-TLBO and MOGWO converge too slowly in the 10-D search space within 10,000 evaluations. At the bottom are the methods that are structurally unable to cover a concave front or lose diversity: Weighted Sum (2 end points, HV 0.210), MODE (1 point, the strict dominance-replacement rule drives every member to the end point (0,1)), NPGA (9 points) and VEGA (1–2 points: sub-populations favour the extremes of each objective).

D1(b) P5 cantilever beam (constrained)

rank by HV	algorithm	HV (mean ± std)	IGD (mean ± std)	Spacing (mean ± std)	points	time s
1	33 SMS-EMOA	1.196 ± 0.000	0.0099 ± 0.0006	0.0152 ± 0.0008	100	13.0
2	26 MOSA	1.195 ± 0.000	0.0049 ± 0.0002	0.0049 ± 0.0012	100	0.8
3	27 MODE	1.195 ± 0.000	0.0048 ± 0.0001	0.0045 ± 0.0012	100	0.6
4	24 MOABC	1.195 ± 0.000	0.0047 ± 0.0002	0.0041 ± 0.0007	100	0.5
5	16 SPEA2	1.195 ± 0.000	0.0050 ± 0.0001	0.0031 ± 0.0003	100	3.1
6	21 Constrained NSGA-II	1.195 ± 0.000	0.0063 ± 0.0003	0.0076 ± 0.0006	97	0.3
7	22 MOPSO	1.195 ± 0.000	0.0074 ± 0.0003	0.0081 ± 0.0005	100	0.4
8	28 MO-TLBO	1.195 ± 0.000	0.0305 ± 0.0130	0.0143 ± 0.0067	100	0.4
9	30 MOMA	1.195 ± 0.000	0.0064 ± 0.0005	0.0075 ± 0.0009	96	0.3
10	23 MOGWO	1.194 ± 0.000	0.0069 ± 0.0005	0.0111 ± 0.0025	100	0.3
11	29 MOBA	1.194 ± 0.001	0.0112 ± 0.0006	0.0070 ± 0.0009	100	0.9
12	17 PESA-II	1.194 ± 0.001	0.0094 ± 0.0009	0.0088 ± 0.0008	100	1.1
13	32 IBEA	1.194 ± 0.000	0.2018 ± 0.0204	0.0316 ± 0.0047	89	0.7
14	11 MOGA	1.193 ± 0.001	0.0132 ± 0.0025	0.0175 ± 0.0030	60	0.1
15	19 MOEA/D-DE	1.193 ± 0.000	0.1209 ± 0.0082	0.0814 ± 0.0031	100	1.0
16	18 MOEA/D	1.193 ± 0.000	0.1208 ± 0.0066	0.0817 ± 0.0020	100	1.2
17	12 NSGA	1.192 ± 0.001	0.0126 ± 0.0012	0.0173 ± 0.0058	63	0.1
18	13 NPGA	1.191 ± 0.001	0.0167 ± 0.0022	0.0292 ± 0.0076	45	0.8
19	14 NSGA-II	1.189 ± 0.002	0.0088 ± 0.0012	0.0472 ± 0.0151	52	0.3
20	35 R-NSGA-II	1.188 ± 0.004	0.0190 ± 0.0069	0.0349 ± 0.0123	75	0.9
21	4 Weight Metric (p-norm, Tchebycheff) *	1.188 ± 0.000	0.3078 ± 0.0000	0.0438 ± 0.0000	21	0.0
22	1 Weighted Sum *	1.183 ± 0.000	0.1859 ± 0.0000	0.2477 ± 0.0000	18	0.0
23	2 eps-Constraint *	1.150 ± 0.000	0.2318 ± 0.0000	0.1935 ± 0.0000	21	0.0
24	10 VEGA	1.143 ± 0.018	0.0756 ± 0.0516	0.0125 ± 0.0071	42	0.0

* classical sweeps (21 weights / eps / Tchebycheff weights) are deterministic up to multi-start, so std = 0 and no time measured per run. True-front HV for reference: 1.194.

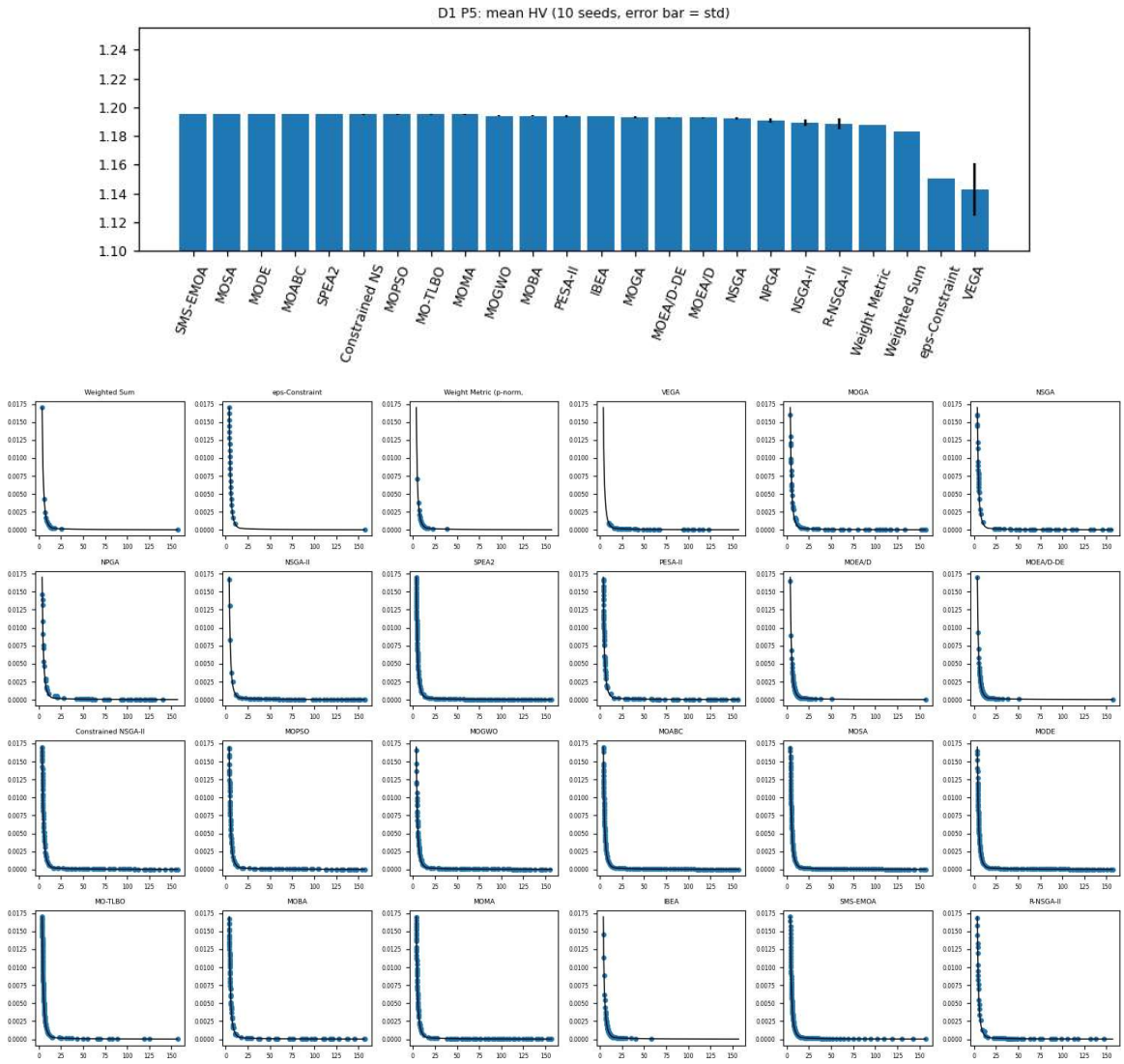
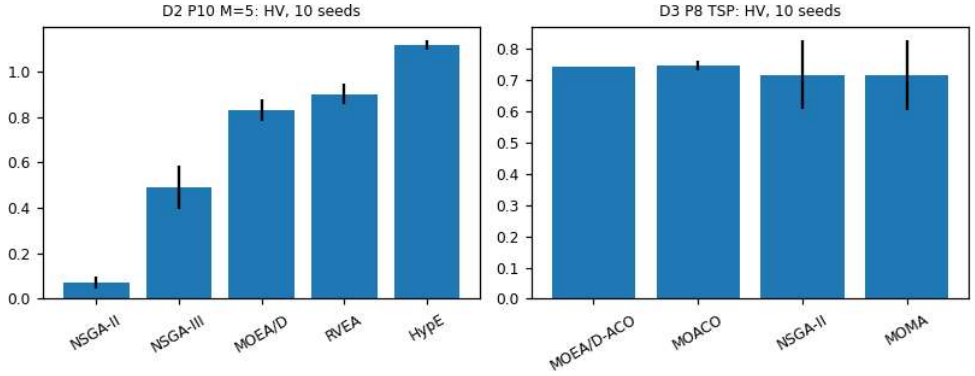


Fig. D-1b: fronts of all algorithms on P5, seed 1 (black line = true front).

Why this ranking (P5). HV saturates (1.195 of 1.194 possible for ten methods), so the differences in the third decimal are tiny and IGD/Spacing are the more informative columns. SMS-EMOA has the top HV but a worse IGD (0.0099); the best IGD (0.0047–0.0050) belongs to MOABC, MODE, MOSA and SPEA2, which also give 100 evenly spread feasible points. Constrained NSGA-II keeps 97 points with Spacing 0.0076, whereas plain NSGA-II, which ignores the constraint during selection, ends with only 52 points and Spacing 0.047 because many of its non-dominated members violate the stress limit and are removed at the end (HV 1.189 vs 1.195). MOEA/D, MOEA/D-DE and IBEA have good HV but IGD 0.12–0.20: the true front of the beam is extremely skewed (mass spans 3.8–157 kg, deflection falls by a factor of 700), so uniform weight vectors or the epsilon-indicator concentrate the points in one part of it. The weight/eps sweeps (0.14–0.31 IGD) suffer from the same effect: equally spaced weights or eps values are very unevenly spread on this front. VEGA is last (1.143) again because it favours the extremes.

D2 Many-objective: DTLZ2 with M = 5 (population 70, 10,000 evaluations, 10 seeds)

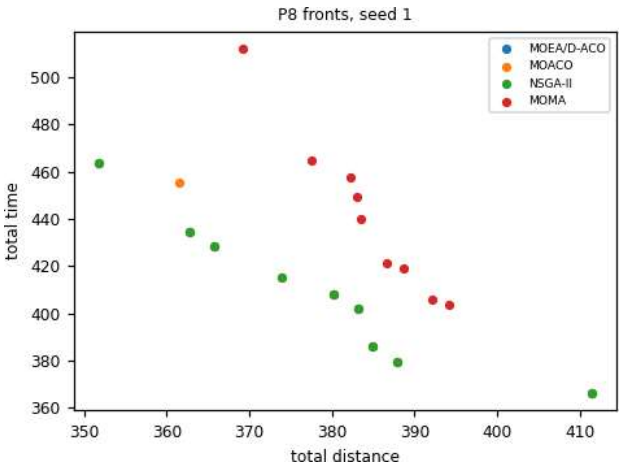
algorithm	HV (mean ± std)	IGD (mean ± std)	Spacing	points	time s
HypE	1.119 ± 0.020	0.306 ± 0.006	0.086 ± 0.015	69.4	59.2
RVEA	0.901 ± 0.045	0.336 ± 0.020	0.107 ± 0.014	45.0	0.1
MOEA/D	0.833 ± 0.048	0.442 ± 0.027	0.181 ± 0.031	33.9	3.5
NSGA-III	0.490 ± 0.095	0.439 ± 0.031	0.139 ± 0.046	69.0	0.7
NSGA-II	0.070 ± 0.027	0.687 ± 0.034	0.132 ± 0.011	69.1	0.4



Ranking and what degrades in NSGA-II. HypE (HV 1.119) > RVEA (0.901) > MOEA/D (0.833) > NSGA-III (0.490) >> NSGA-II (0.070); IGD gives the same order except that NSGA-III and MOEA/D are tied (0.439 vs 0.442), MOEA/D converges better (HV) but ends with only 33.9 distinct points. NSGA-II is the clear loser: with 5 objectives about 57% of a random DTLZ2 population is already non-dominated (35% at M = 3, 70% at M = 8; Section 8, P10), so non-dominated sorting hardly distinguishes individuals; after the first front is full, selection falls back on crowding distance, which in 5-D rewards isolated points that can be far from the front. All 69 points in the final NSGA-II population are mutually non-dominated yet IGD is 0.687, i.e. the set is spread but not converged. NSGA-III, RVEA and MOEA/D replace crowding by reference vectors or weights and keep the population aligned with the simplex; HypE selects by (estimated) hypervolume contribution, which still discriminates between non-dominated points, but costs 59 s per run against 0.1 s for RVEA. Caveat: at 10,000 evaluations (about 140 generations) every method is still far from the front (IGD 0.31–0.69); NSGA-III in particular is under-converged and overtakes NSGA-II and MOEA/D at larger budgets (30,000 evaluations on M = 3: IGD 0.055 vs 0.092 and 0.078).

D3 Discrete: P8 bi-objective TSP (15 cities), permutation operators (10 seeds)

algorithm	HV (mean ± std)	Spacing	points	time s
MOACO	0.744 ± 0.015	0.103 ± 0.003	10.0	7.2
MOEA/D-ACO	0.741 ± 0.001	0.127 ± 0.010	8.2	7.6
NSGA-II	0.716 ± 0.111	0.098 ± 0.029	9.3	1.0
MOMA	0.714 ± 0.111	0.103 ± 0.032	9.2	0.7



Ranking. MOACO (HV 0.744 ± 0.015) and MOEA/D-ACO (0.741 ± 0.001) are statistically tied, followed by NSGA-II (0.716 ± 0.111) and MOMA (0.714 ± 0.111), also tied. The two ant methods are consistent from seed to seed (std 0.001–0.015) because construction guided by 1/distance and 1/time already gives good tours, but they keep only 8–10 distinct tours. The permutation GAs (order crossover + inversion/swap) usually reach the same quality with similar numbers of points but with a large standard deviation (0.111): in some seeds the population stalled early. MOMA's local search did not pay off here (0.714 vs 0.716) because the local-search evaluations are taken from the same 10,000-evaluation budget. With only 15 cities the

problem is small, so the differences between methods are small; the benefit of the pheromone heuristics should grow with problem size.

D4 Methods that do not produce a front on their own

Algorithms 1–4 were swept with 21 weights / eps values / Tchebycheff weights (15 reference points for Benson) and appear in the D1 tables. **Algorithms 5–7**: the preference or priority that produced each answer is listed in Section 3 (value-function exponents a for investor and laptop profiles; goal weights and targets; priority order and tolerance).

TOPSIS and VIKOR on the NSGA-II front of P4 (97 portfolios, weights 0.5 risk / 0.5 return, then 0.6 / 0.4):

method	top 3 (risk, return) with weights 0.5/0.5	top 3 after shifting weight 0.1 (0.6/0.4)	winner flips?
TOPSIS	1. (0.0127, 0.1130) w=[0.191, 0.284, 0.223, 0.302] 2. (0.0122, 0.1122) w=[0.196, 0.287, 0.207, 0.31] 3. (0.0139, 0.1150) w=[0.173, 0.29, 0.253, 0.283]	1. (0.0113, 0.1103) w=[0.227, 0.286, 0.182, 0.305] 2. (0.0106, 0.1088) w=[0.239, 0.23, 0.18, 0.351] 3. (0.0101, 0.1074) w=[0.315, 0.277, 0.164, 0.244]	**yes**
VIKOR (v=0.5)	1. (0.0322, 0.1330) w=[0.0, 0.416, 0.494, 0.09] 2. (0.0327, 0.1334) w=[0.0, 0.466, 0.481, 0.053] 3. (0.0299, 0.1314) w=[0.0, 0.509, 0.425, 0.066]	1. (0.0262, 0.1289) w=[0.0, 0.466, 0.392, 0.142] 2. (0.0268, 0.1292) w=[0.009, 0.486, 0.394, 0.111] 3. (0.0253, 0.1281) w=[0.0, 0.484, 0.369, 0.147]	**yes**

P12 suppliers: Pareto-optimal suppliers ['S1', 'S2', 'S3', 'S4', 'S5'] (S4 is cheapest, S3 best on quality/delivery/reliability, S1 and S5 balanced; S2 is dominated by none either). TOPSIS: ['S3', 'S1', 'S5', 'S2', 'S4']; with weight shifted 0.1 from cost to quality: ['S3', 'S5', 'S1', 'S2', 'S4'] → **the ranking flips (S1 and S5 swap)**. VIKOR v = 0.25 / 0.5 / 0.75: ['S1', 'S2', 'S5', 'S3', 'S4'] / ['S1', 'S2', 'S5', 'S3', 'S4'] / ['S1', 'S3', 'S2', 'S5', 'S4']; after the shift at v = 0.5: ['S1', 'S3', 'S5', 'S2', 'S4']. When S3's cost is raised to 400 TOPSIS ranks ['S1', 'S5', 'S2', 'S4', 'S3'] and VIKOR (v = 0) ['S1', 'S5', 'S2', 'S3', 'S4']: both punish S3, VIKOR ties it with S4 on maximal regret. A cleaner test is a toy case of two alternatives with 4 equally weighted benefit criteria, A = (6,6,6,6) and B = (10,10,10,1): TOPSIS C = 0.598 vs 0.577 (A narrowly first), VIKOR v = 0 gives Q = 0 vs 1 (A decisively first) and v = 1 gives Q = 0.275 vs 0 (B first). **VIKOR with a small v protects against one very bad criterion** because R is the largest single regret; TOPSIS lets the three excellent criteria compensate.

Algorithms 36–37 on P13

setting	method	final f1 or L_cls/L_cls(0)	final f2 or L_reg/L_reg(0)	final weights
P13a toy (losses, start (0,5))	equal	18.000	18.000	(1,1)
P13a toy (losses, start (0,5))	mgda	0.500	60.500	alpha* from min-norm solver
P13a toy (losses, start (0,5))	gradnorm	5.601	37.437	[1.63, 0.37]
P13b network, regression target x1	equal	0.127	0.009	[1.0, 1.0]
P13b network, regression target x1	mgda	0.163	0.017	[1.0, 1.0]
P13b network, regression target x1	gradnorm	0.097	0.048	[1.85, 0.15]
P13b network, regression target x10	equal	0.439	0.015	[1.0, 1.0]
P13b network, regression target x10	mgda	0.158	0.019	[1.0, 1.0]
P13b network, regression target x10	gradnorm	0.246	0.112	[1.99, 0.01]

Reading the gradient results. All methods reduce both losses (every ratio is below 1). In the toy problem equal weights go to the midpoint (0,0), the best compromise in sum (losses 18 + 18), whereas MGDA stops as soon as it reaches the Pareto segment at (2.5, 2.5), the projection of the start (0,5) onto it (losses 0.5 + 60.5): MGDA guarantees a Pareto point, not a balanced one. GradNorm ends at (1.33, 1.33) with weights (1.63, 0.37). In the network with equal task scales GradNorm lowers the classification loss most (0.097 of its initial value) but is worse on regression (0.048 vs 0.009 for equal weights). When the regression targets are scaled by 10 its gradients dominate: fixed equal weights let the regression task take over and the classification loss only reaches 0.439 of its start; MGDA with normalised gradients keeps both low (0.158 / 0.019); GradNorm drives the regression weight to its floor (w = (1.99, 0.01)) and over-corrects (regression loss 0.112, classification 0.246). Hence MGDA gave the most balanced result and GradNorm the most aggressive re-balancing.

7. Task E — algorithm selection reasoning (six problems)

Problem	Two most suitable	One poor choice	Justification (convexity, M, variables, constraints, preference, budget)	Evidence from my runs
P3 ZDT2 (non-convex, 2 obj, 10 real vars, bounds only)	MOEA/D-DE; PESA-II (or MOABC)	Weighted Sum	Non-convex front: a linear scalarisation can only select supporting points, so it returns the two end points; dominance- or Tchebycheff-based search is needed. Two objectives and cheap evaluations favour a population method with a strong operator (DE) and an archive.	MOEA/D-DE HV 0.531±0.0037, IGD 0.0060±0.0014; PESA-II HV 0.527±0.0025; Weighted Sum HV 0.210±0.000 with 2 points (Fig. D-1a, Section 6).
P5 cantilever beam (constrained, 2 vars, strongly skewed front)	Constrained NSGA-II; SPEA2 (or MODE)	MOEA/D-DE (uniform weights) / ignoring the constraint	Hard stress constraint: Deb's rules rank feasible designs first without a penalty factor; the front spans orders of magnitude, so methods whose diversity comes from uniform weights or from sweeps of eps/weights spread poorly, while density estimators (crowding, k-th neighbour) adapt.	Constrained NSGA-II 97±1 points, Spacing 0.0076±0.0006, IGD 0.0063±0.0003; SPEA2 IGD 0.0050±0.0001; MOEA/D-DE IGD 0.121±0.0082, Spacing 0.081±0.0031; plain NSGA-II keeps only 52±3 points and 45% of the unconstrained front violates the stress limit (Section 8).
P10 DTLZ2, M = 5 (many-objective, real vars)	RVEA; HypE (NSGA-III when more evaluations are available)	NSGA-II	With 5 objectives most random points are non-dominated (57%), so Pareto rank has little pressure and crowding distance loses meaning; reference vectors/points (RVEA, NSGA-III) or hypervolume contributions (HypE) restore selection pressure. RVEA is the cheapest, HypE the most accurate but about 500 times slower.	HV: HypE 1.119±0.020, RVEA 0.901±0.045, NSGA-III 0.490±0.095, NSGA-II 0.070±0.027; time RVEA 0.1 s vs HypE 59.2 s (Fig. D-2).
P8 bi-objective TSP (discrete, permutations)	MOACO; MOEA/D-ACO (NSGA-II with permutation operators is a close third)	Weighted Sum / eps-constraint (and any real-coded method without a decoder)	Permutation encoding: order crossover and inversion keep tours valid (one-point crossover produced invalid tours); pheromone construction with distance/time heuristics uses problem knowledge. Gradient-based scalarisation does not apply to discrete tours.	MOACO HV 0.744±0.015, MOEA/D-ACO 0.741±0.0007, NSGA-II 0.716±0.111 (std 0.11), one-point crossover invalid: Section 8 (P8).
P11 production planning (linear goals / priorities)	Goal Programming; Lexicographic	NSGA-II (or any stochastic EA)	A two-variable LP with explicit targets and priorities is solved exactly by an LP solver in milliseconds; the profit tie along an edge is resolved by the second priority. An EA would only approximate the vertex and cannot certify that the units goal is unreachable by exactly 100.	GP: x = (300,200), shortfall 100 (exactly the minimum); Lexicographic: profit 2400 then units 500 (Fig. A-06, A-07).
P14 laptop (preference: 1.8 kg, 12 h)	R-NSGA-II; Value Function	Lexicographic (or VEGA)	The decision maker gave an aspiration, not a priority; a preference-based EA concentrates its evaluations around the aspiration while still returning alternatives; a value function returns one answer per profile. Lexicographic would force a strict priority that the manager did not give.	R-NSGA-II: 69% of the points within 0.15 kg of 1.8 kg vs 46% for NSGA-II (Fig. E-5).

The answers cover a many-objective problem (P10), constrained problems (P5), a discrete problem (P8), a non-convex problem (P3) and preference problems (P11, P14); five of the six are backed by Task A or D results (Section 6).

8. Answers to the 'Investigate' questions of P1–P14

P1 — factory speed

All 21 weights gave 21 distinct Pareto points for weighted sum and for every p; so the number of distinct points equals the number of weights. Equal steps in weight do **not** give equal spacing on the front: for weighted sum the nearest-neighbour step in normalised objective space ranges between 0.071 and 0.098, for p = 2 between 0.023 and 0.126, and for p = inf between 0.034 and 0.149 (Spacing 0.0080 for p = 1 and weighted sum, 0.0300 for p = 2, 0.0335 for p = inf): for p = 2 and inf the points crowd around the middle (x between 4.8 and 5.2 holds 7 of the 21 points for p = 2) and the end points are far apart. Weighted sum and p = 1 produce the same solutions (x = 7.0, 6.8, … 3.0) because on a convex front the p = 1 metric is the weighted sum.

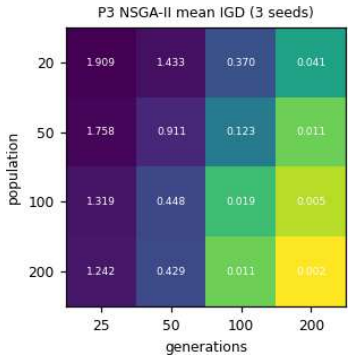
P2 — fleet sizing (every x is Pareto-optimal)

Real x: NSGA-II returns 98 distinct x with quartiles [1.0, 1.97, 4.33, 7.56, 10.0]; VEGA also 98 distinct points but its quartiles [1.0, 3.46, 5.14, 6.81, 9.95] show a less even coverage (Spacing 0.038 vs 0.0063 for NSGA-II). Neither clusters at one end: NSGA-II puts 38% of its points at x < 3, where f1 = 100/x changes steeply, so it is even in objective space but not in x; the eps-constraint sweep with equal eps steps gives x = 1.0, 2.44, 3.30, 3.98, … 10 (dense at large x, sparse at small x) and Spacing 0.129. Integer x: the front has exactly 10 points (x = 1…10); both VEGA and NSGA-II find all 10 (HV 1.0873, Spacing 0.128, the same for both), so an integer variable replaces 98 near-continuous points by 10 well separated ones and the Spacing metric jumps from 0.006–0.04 to 0.128.

P3 — ZDT2: missing methods and IGD vs population / generations

Weighted Sum (for any of 21 weights, only the points (0,1) and (1,0)) and p = 1 never find the middle of the front; Tchebycheff (p = inf), eps-constraint, Benson with an attainable reference point and every Pareto-based EA do. IGD of NSGA-II (mean of 3 seeds, population × generations, budget = pop × (gens + 1)):

population \ generations	25	50	100	200
20	1.9088	1.4331	0.3695	0.0409
50	1.7579	0.9112	0.1231	0.0105
100	1.3192	0.4485	0.0193	0.0051
200	1.2417	0.4295	0.0107	0.0025



More generations always help (pop 100: 1.319 → 0.448 → 0.0193 → 0.0051 for 25/50/100/200 generations); more population only helps when there are enough generations (pop 200 with 25 generations is worse, 1.242, than pop 50 with 25 generations, 1.758, because too few generations remain for convergence), and a small population (20) is limited by the number of points it can hold even with many generations (0.041 at 200 generations).

P4 — portfolio: repair vs penalty and investor profiles

Repair by normalisation (w = x / Σx) gives always feasible weights and the best results: HV 0.9664 ± 0.0003, IGD 0.0056 ± 0.0004. The penalty version (violation |Σx - 1| - 0.01, selection by penalty) ends with HV 0.888 ± 0.051 and IGD 0.060 ± 0.033, and its solutions violate the equality by up to 0.0100, i.e. the equality is only approximately satisfied and the search spends effort on feasibility. I therefore chose repair: cheap, exact, and it keeps the search on the simplex (its drawback is that x and c·x map to the same portfolio, which is harmless for the front).

point on the NSGA-II front	risk	return	weights (A1..A4)	profile
conservative(min-risk end)	0.0070	0.0927	[0.609, 0.116, 0.051, 0.224]	conservative: mostly the safest asset A1
conservative-balanced	0.0161	0.1184	[0.087, 0.331, 0.271, 0.311]	conservative-balanced
balanced	0.0338	0.1340	[0.0, 0.448, 0.501, 0.051]	balanced: A2/A3 mixture
aggressive-balanced	0.0606	0.1438	[0.002, 0.198, 0.797, 0.003]	aggressive-balanced: mostly A3
aggressive(max-return end)	0.0900	0.1500	[0.0, 0.0, 1.0, 0.0]	aggressive: all in A3 (return 0.15, risk 0.09 - matches the sanity check)

P5 — cantilever: infeasible fraction and ignoring the stress limit

2.6% of 100,000 random designs are infeasible. Sanity design (b,h) = (0.01, 0.05): mass 3.925 kg, deflection 0.016 m, stress 240 MPa (feasible). I use Deb's rules (Constrained NSGA-II) for the final comparison and a normalised-violation penalty for the other algorithms. Ignoring the stress limit makes the front unrealistically good at the light end (minimum mass 1.57 kg vs 3.85 kg with the limit) and **45% of those points violate the stress limit**; they are not usable designs.

P6 — Binh-Korn: Deb's three rules by hand

candidate	(x1, x2)	f1	f2	constraint check	CV
a	(1.0, 1.0)	8.00	32.00	feasible	0.00
b	(4.0, 2.0)	80.00	10.00	feasible	0.00
c	(0.0, 3.0)	36.00	29.00	(0-5)^2+3^2 = 34 > 25: violated by 9 (CV = 9/25)	0.36
d	(0.5, 3.0)	37.00	24.25	(0.5-5)^2+3^2 = 29.25 > 25: violated by 4.25 (CV = 4.25/25)	0.17

Rule 1 (feasible beats infeasible): a and b beat c and d whatever their objectives. Rule 2 (both infeasible: smaller violation wins): d (0.17) beats c (0.36) although c has a smaller f1 (0 vs 2). Rule 3 (both feasible: ordinary dominance): a (8, 32) and b (80, 8.. see table) are incomparable. Pairwise result of my code: a vs b: incomparable; a vs c: **a wins**; a vs d: **a wins**; b vs c: **b wins**; b vs d: **b wins**; c vs d: **d wins**. Without feasibility rules (plain NSGA-II, final population feasible in 100% of 5 runs, 97.6 non-dominated points) the algorithm still returns feasible points on P6, because the Pareto front lies away from the violated region of the box; on P5, where the front touches the constraint, it does not (Section 6).

P7 — water allocation

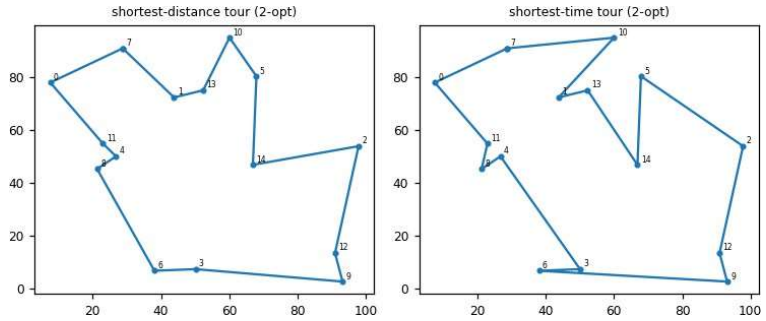
Maximisations were converted to minimisation (-0.8 x1, -1.2 x2) before every dominance test. Constrained NSGA-II's 82 points lie on the line x1 + x2 = 90 within 0.95 million m3 (the front sits on the constraint boundary, so the feasibility rules pull points onto it from inside; the 10,000-evaluation front is not exactly on the line). A decision maker at (40, 50) prefers the point on the line closest to the reference: x1 = 39.9, distance 0.0024 in normalised units, i.e. essentially the reference itself (f = (32, 60)); Goal Programming hits it with zero deviation, and R-NSGA-II concentrates its points at x1 quartiles [12.28, 38.78, 39.89, 41.59, 64.56] versus NSGA-II [10.69, 24.54, 39.89, 54.9, 68.36].

P8 — bi-objective TSP

Coordinates used (numpy.random.seed(7), 15 uniform points in [0,100]^2, then u ~ U(0.5,2) symmetrised): 0:(7.6,78.0); 1:(43.8,72.3); 2:(97.8,53.9); 3:(50.1,7.2); 4:(26.8,50.0); 5:(67.9,80.4); 6:(38.1,6.6); 7:(28.8,91.0); 8:(21.3,45.2); 9:(93.1,2.5); 10:(60.1,95.0); 11:(23.0,54.8); 12:(90.9,13.3); 13:(52.3,75.0); 14:(66.9,46.8).

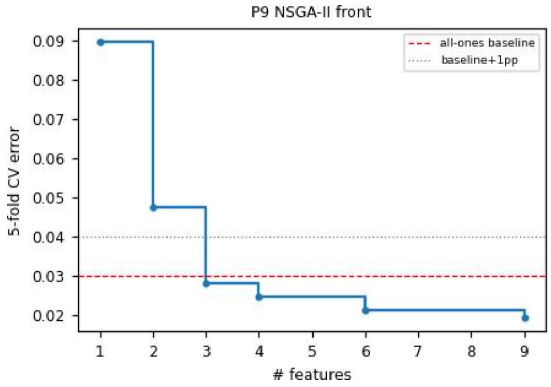
tour	distance	time
random tour (mean of 2000)	817.1	1001.0
nearest neighbour on distance	362.0	484.1
nearest neighbour on time	490.4	495.7
2-opt shortest-distance tour	351.8	463.7
2-opt shortest-time tour	411.4	366.3

The shortest-distance and shortest-time tours share only 36% of their edges (Jaccard 0.36), so the two objectives really conflict: the shortest-distance tour needs 463.7 time units (vs 366.3 minimum, +27%) and the shortest-time tour 411.4 distance units (vs 351.8, +17%). Nearest-neighbour is 3% above the 2-opt distance but 35% above the 2-opt time, because the time matrix is less geometric. Operators (NSGA-II, 5 seeds, HV): OX + inversion + swap: 0.754 ± 0.005; OX + inversion: 0.755 ± 0.001; OX + swap: 0.756 ± 0.001; inversion only (no crossover): 0.530 ± 0.185; OX only (no mutation): 0.558 ± 0.153. Order crossover with either inversion or swap (or both) works (HV about 0.755, std 0.001–0.005); using only mutation or only crossover is clearly worse and unstable (0.53–0.56, std 0.15–0.18). A one-point crossover on permutations produced an invalid tour (repeated cities) in my test, which is why order crossover is needed.



P9 — feature selection (Wisconsin, 5-NN, 5-fold CV coded by me)

The all-30-features baseline has accuracy 0.9701 (CV error 0.0299). Combining three NSGA-II runs (population 60, 3,000 evaluations each) gives 6 non-dominated subsets: (1, 0.090), (2, 0.047), (3, 0.028), (4, 0.025), (6, 0.021), (9, 0.019) (features, error). **The smallest feature set within 1 percentage point of the baseline accuracy has 3 features** (worst texture, worst area, worst concave points) with error 0.0281 — even better than all features. Most frequent features in the front solutions: worst texture 67%, worst area 67%, mean concave points 50%, worst concave points 50%, worst radius 33%.



P10 — DTLZ2: non-dominated fraction of a random population

Fraction of a random population of 100 that is non-dominated (mean ± std over 30 populations): M = 3: 35% ± 6%, M = 5: 57% ± 8%, M = 8: 70% ± 8% (for uniformly random objective vectors: 15%, 45%, 81%). With many objectives dominance relations become rare, almost everybody is rank 1 and selection pressure from Pareto rank collapses; the survivors are then chosen by the secondary criterion, which is why that criterion (crowding distance, reference points, indicator) decides everything.

P11 — production planning

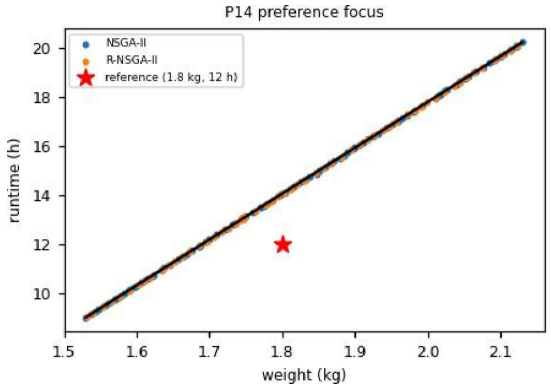
Feasible corners (0,0), (400,0), (300,200), (0,400). Maximum profit 2400 occurs at (300,200) and at (0,400) and everywhere between; maximum units is 500 at (300,200), so the units goal 600 can never be met and Goal Programming leaves a shortfall of exactly 100 (unit-weight objective 100, normalised-weight objective 0.1667 = 100/600, units-heavy weights 1000; the solution is (300,200) in all three). **Why the profit tie must be broken by the second priority:** the profit optimum is not unique (a whole edge), so Lexicographic optimisation needs the second objective to choose among the tied points; units are 400 at (0,400) but 500 at (300,200), so (300,200) is chosen. With units first the answer is also (300,200), where profit 2400 is still attained.

P12 and P13

See Section 6 (D4): rankings and flips for TOPSIS/VIKOR, and the comparison of fixed equal weights with MGDA and GradNorm.

P14 — laptop with a preference

The algorithm discovers that x1 = 13 for all front points: 99% of NSGA-II's final points have x1 within 0.05 of 13 (maximum x1 = 13.068). The reference point (1.8 kg, 12 h) is **behind** the front: the front reaches 14 h at 1.8 kg and gives 12 h already at 1.69 kg, so a front point dominates the reference (it is attainable and improvable). R-NSGA-II focuses on the region around the aspiration: weight quartiles [1.531, 1.745, 1.745, 1.884, 2.124] versus [1.53, 1.665, 1.829, 1.978, 2.13] for NSGA-II, and 69% of its points lie within 0.15 kg of 1.8 kg versus 46%.



9. Task F — viva preparation (answers with evidence)

1. Why can Weighted Sum never reach a dented part of a non-convex front?

A weighted sum w·f is a straight line (hyperplane) in objective space; minimising it means sliding the line inwards until it touches the feasible set, and on a concave (dented) front the first contact is always an end point. Example on ZDT2 with w = (0.5, 0.5): the middle point (0.5, 0.75) scores 0.625, but the ends (0,1) and (1,0) score 0.5, so the end wins for every w. My run found only 2 points (HV 0.210 vs 0.542 true).

2. Why did VEGA collapse towards the extremes in my run?

Each sub-population is selected on one objective only, so individuals that are excellent in one objective but poor in the other are favoured, and compromise points are never selected by anyone. After shuffling and mating the population converges to the best individuals of each objective. On ZDT2 the final non-dominated set had 1 point(s) (HV 0.000, 10 seeds: 0.000).

3. What does elitism guarantee, and which of my algorithms have it?

That the best non-dominated solutions found so far are never lost, so the hypervolume of the retained set cannot decrease (checked in unit test 14). Elitist: NSGA-II (14), NSGA-III (15), SPEA2 (16), PESA-II (17), MOEA/D variants (18–20, neighbour replacement only if not worse), Constrained NSGA-II (21), MOPSO/MOGWO/MOABC/MOACO (archive, 22–25), MOSA/MODE/MO-TLBO/MOBA (via archive, 26–29), MOMA (30), IBEA/SMS-EMOA/HypE/R-NSGA-II (32–35). Not elitist: VEGA, MOGA, NSGA, NPGA (10–13); their result is only the last population.

4. Why does crowding distance stop working well with many objectives, and what replaces it?

With M = 5 about 57% of a random population is already non-dominated (Section 8), so almost everybody is in the first front and the cut is made by crowding distance, which sums per-objective neighbour gaps; in many dimensions nearly every point is far from its neighbours in some objective, the measure rewards isolated/boundary points, and does not indicate closeness to the front. NSGA-II gets HV 0.070 vs 0.490 for NSGA-III and 0.901 for RVEA (Fig. D-2). It is replaced by reference points (NSGA-III), reference vectors with angle-penalised distance (RVEA), decomposition weights (MOEA/D), or indicators (HypE, SMS-EMOA, IBEA).

5. Which algorithms need a preference from the decision maker, and what happens if it is wrong?

Weighted Sum and Weight Metric (weights), eps-Constraint (eps), Benson (reference point and weights), Value Function (utility), Goal Programming (targets), Lexicographic (priorities), R-NSGA-II (reference points), TOPSIS/VIKOR (weights, v) and MOEA/D (weight vectors, implicitly). If wrong: a different utility profile moves P4 from the 0.021-risk to the 0.040-risk portfolio; priorities reversed on P7 give (10,80) instead of (70,20); a Benson reference point beyond the nadir degenerates to weighted sum and one that is not attainable makes the problem infeasible (Fig. C-1); an R-NSGA-II reference far from the front yields points near the closest part of the front, not near what is wanted; a 0.1 weight shift swaps S1 and S5 in TOPSIS.

6. Explain Deb's three constraint rules with one numerical example.

(1) A feasible solution beats any infeasible one; (2) of two infeasible solutions the one with the smaller total violation wins; (3) of two feasible solutions the one that Pareto-dominates wins (otherwise they are incomparable). On Binh-Korn (Section 8, P6): a = (1,1) feasible, c = (0,3) with CV = 0.36 and d = (0.5,3) with CV = 0.17: a beats c and d by rule 1 although c has f1 = 36 vs a's 8 (rule 1 ignores objectives), d beats c by rule 2 although c's f1 is smaller, and a and b = (4,2) are incomparable by rule 3.

7. How would I detect that my algorithm has prematurely converged?

Track the hypervolume (or IGD) of the non-dominated set per generation and stop/alarm when it is flat for many generations while the set is small or far from the front; track the number of distinct non-dominated points and Spacing, and the spread of the population in decision space. In my runs MODE on ZDT2 reached its final HV 0.110 by generation 80 with a single point, and VEGA stays at a flat value from generation 5, whereas NSGA-II keeps improving until generation 95 (Fig. E-2).

8. Why do TOPSIS and VIKOR not count as optimisers?

They take a finite, given list of alternatives and only rank it (by closeness to ideal/anti-ideal or by regret); they do not search the decision space, generate new alternatives or improve any of them. They are used after an optimiser (here applied to the NSGA-II front of P4 and to the P12 suppliers); their result also depends on the chosen weights and, for VIKOR, v.

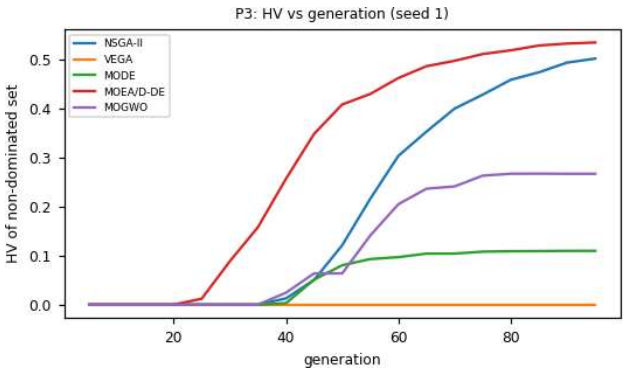


Fig. E-2: hypervolume of the non-dominated set against generation on ZDT2 (seed 1).

10. Reproducibility and files

Everything is plain Python 3 with NumPy and Matplotlib (scipy only inside algorithms 1–7 and for generating the P4 reference front with my own eps-constraint sweep). Seeds: Task A seed 1; Tasks C and D seeds 1–10; extras seeds 1–5 as stated; P8 uses numpy.random.seed(7); unit tests use fixed seeds. Run order: `python tests/test_all.py; cd experiments; python run_task_a.py; python run_task_c.py; python run_task_d.py; python run_extras.py; python make_figures.py; python build_report.py`. Typical runtimes on one CPU core: Task A about 4 min, Task C about 6 min, Task D about 45 min (HypE on M = 5 is the slowest part, 59 s per run), extras about 6 min.

File	Content
code zip	moo/ (moo_utils, problems, ops, base, 9 algorithm modules), tests/test_all.py, experiments/*.py, notebook, README
outputs zip	figures/ (PNG), data/ (taskA.json, taskA_fronts.npz, taskC.json, taskD.json, taskD_fronts.npz, extras.json, unit_tests.json and logs), the report PDF
notebook	Lab7_Manavaditya_Rajesh_241070040.ipynb - runs the tests and a condensed version of the experiments with the same code

Academic integrity: all algorithms and metrics are my own code; no optimisation library is used. Explanations and conclusions are based on my own experiments.